MANUAL TÉCNICO — TACTICALGRID

1. Portada

Campo	Valor
Nombre del proyecto	TacticalGrid
Descripción corta	Escaramuza táctica 2D por turnos — Sistemas Inteligentes I
Integrantes del equipo	• Sophia Cardona • Danna Alexandra Madrid Roa • Michel Dahiana Arenas López
Asignatura / Programa	Sistemas Inteligentes I — Ingeniería de Sistemas y Computación — Universidad de Caldas
Versión del proyecto	1.1 (Octubre 2026)
Versión del formato de escenario	`1.0` (campo `version` de los archivos JSON; no es la versión del software)
Rama Git analizada	`main`
Último commit analizado	`ae0664c` — «Modifica una linea» — 2026-10-20 23:20:29 -0500 (los commits previos `b713b72` e `3f1fa40` integran la interfaz y el módulo de Minimax/alfa-beta)
Fecha de elaboración del manual	2026-10-19
Fecha de última actualización	2026-10-20 (incorpora Beam Search, Minimax/alfa-beta, pruebas y resultados experimentales nuevos)
Elaboración	Generado a partir del código fuente, archivos JSON, pruebas y documentación existentes
Interprete verificado en el entorno de elaboración	Python 3.12.0 (versión inicial) y Python 3.14.0 (versión de la re-ejecución de pruebas y experimentos del 2026-10-20)

> Este documento se basa exclusivamente en los archivos presentes en el repositorio. Todo dato no verificable se marca expresamente. La actualización del 2026-10-20 se realizó sobre el commit `ae0664c` más los cambios sin commitear indicados en la sección 15.2. Durante esta actualización se modificaron únicamente dos archivos de prueba/experimentación (no la lógica de los algoritmos): `tests/test_beam_search.py` (el criterio de éxito de `k=2` se alineó con el comportamiento real medido) y `experimentos/ejecutar_experimentos_busqueda.py` (se añadieron los casos robustos y adversariales).

2. Tabla de contenido

1. Portada
2. Tabla de contenido
3. Introducción, objetivo y alcance técnico
4. Tecnologías, lenguajes, frameworks, bibliotecas y herramientas
5. Estructura de carpetas y responsabilidad de archivos
6. Arquitectura del sistema
7. Modelado del problema, representación de estados y modelo de datos
    - 7.1 Formalización de Navegación (Tupla Canónica)
    - 7.2 Formalización del Juego Táctico por Turnos (Estado Compuesto)
    - 7.3 Comparación de Espacios y Tamaño Combinatorio (|S|)
    - 7.4 Entidades Internas y Clases de Python
8. Descripción técnica de clases, funciones y métodos
    - 8.5 Heurísticas y Demostración Formal (Admisibilidad, Consistencia y Dominancia)
    - 8.6 Beam Search (Búsqueda en Haz con memoria acotada `k`)
    - 8.7 Minimax y poda Alfa-Beta (búsqueda adversarial)
9. Parámetros, tipos, retornos, excepciones y efectos secundarios

10. Configuración, instalación y ejecución
11. Pruebas existentes y mecanismos de validación
12. Metodología experimental
13. Resultados cuantitativos y experimentales
14. Análisis teórico y empírico de complejidad
15. Limitaciones técnicas, deuda técnica y funcionalidades pendientes
16. Reporte del uso de inteligencia artificial
17. Anexos

3. Introducción, objetivo y alcance técnico

3.1 Introducción

TacticalGrid es una aplicación de escritorio desarrollada en Python que integra, en una misma interfaz gráfica basada en `tkinter`:

- la **carga y validación** de escenarios de juego definidos en JSON;
- la **simulación por turnos** de una escaramuza táctica 2D sobre una cuadrícula;
- la **resolución de problemas de navegación** mediante algoritmos de búsqueda;
- la **verificación** de los caminos producidos por dichos algoritmos;
- la **comparación** de métricas de algoritmos y heurísticas.

El sistema está organizado en capas: modelo de escenario, modelo de juego y reglas, algoritmos de búsqueda, verificación e interfaz. La interfaz no contiene reglas del juego ni algoritmos; los invoca a través de módulos especializados (véase `interfaz/app.py`, línea 1).

3.2 Objetivo

Proveer una herramienta académica para:

1. Modelar un escenario táctico en una cuadrícula con terrenos de distinto costo.
2. Ejecutar y comparar algoritmos de búsqueda (BFS, DFS, UCS, A* y Beam Search).
3. Verificar automáticamente que un camino hallado cumple las propiedades exigidas.
4. Simular una partida por turnos con reglas deterministas (movimiento, recogida e interceptación de un recurso, condición de victoria).
5. Resolver decisiones adversariales con Minimax y poda alfa-beta, tanto de forma aislada como como controlador de un bando en la partida.

3.3 Alcance técnico verificado

Funcionalidad	Estado observado en el código
Carga de escenarios JSON con validación completa	Implementada
Modelo de estado inmutable y hashable	Implementada
Reglas de movimiento, recogida e interceptación	Implementadas
Condición de victoria	Implementada
BFS, DFS	Implementados
UCS (Dijkstra)	Implementado
A* con heurísticas `manhattan`, `propia` y `euclidiana`	Implementado
Beam Search (`k`)	**Implementado**: registrado como `BEAM_SEARCH`; probado con `k = 1, 2, 4, 8`
Verificación de caminos	Implementada
Interfaz gráfica con cinco pestañas	Implementada
Comparación de todos los algoritmos de búsqueda	Implementada
Comparación de heurísticas de A* en Estadísticas	Implementada
Exportación del historial a CSV	Implementada

Funcionalidad	Estado observado en el código
Partida por turnos con control Humano, «Azar», Minimax y Alfa-Beta	Implementada
Tarea adversarial (Minimax / alfa-beta)	**Implementada**: evalúa el estado, compara ambos algoritmos y aplica la jugada

La aplicación no utiliza servicios de red, bases de datos ni almacenamiento persistente más allá de la lectura de archivos JSON y la escritura opcional de un CSV.

4. Tecnologías, lenguajes, frameworks, bibliotecas y herramientas

4.1 Lenguaje y versión

Elemento	Evidencia
Lenguaje	Python
Versión mínima declarada	3.9+ (`README.md` , línea 9)
Versión verificada en el entorno	3.12.0 (`python --version` durante la elaboración)

No existe archivo `requirements.txt` , `pyproject.toml` , `setup.py` , `Pipfile` , `environment.yml` ni configuración equivalente en el repositorio (verificado por búsqueda de archivos).

4.2 Bibliotecas utilizadas

Todas las importaciones corresponden a la biblioteca estándar de Python. No se detectaron dependencias de terceros.

Módulo estándar	Uso	Archivos donde se importa
`tkinter` (`tk` , `ttk` , `filedialog` , `font` , `messagebox`)	Interfaz gráfica	`interfaz/app.py` , `interfaz/tablero.py`
`json`	Lectura de escenarios y serialización de resultados	`escenario/cargador.py` , `interfaz/app.py`
`heapq`	Colas de prioridad de UCS y A*	`algoritmos/busqueda_con_costo.py` , `algoritmos/a_estrella.py`
`collections.deque`	Frontera FIFO de BFS	`algoritmos/busqueda_no_informada.py`
`dataclasses`	Entidades inmutables, resultado de búsqueda y resultado adversarial	`escenario/mapa.py` , `escenario/modelos.py` , `juego/estado.py` , `algoritmos/base.py` , `algoritmos/minimax.py`
`typing.Optional`	Anotaciones de tipo	`escenario/modelos.py` , `juego/estado.py` , `algoritmos/base.py`
`time`	Medición de tiempo con `perf_counter`	`algoritmos/base.py` , `algoritmos/minimax.py`
`random`	Controlador «Azar» de la partida y generación de casos	`interfaz/app.py` , `experimentos/generar_casos_minimos.py`
`csv`	Exportación del historial	`interfaz/app.py`
`os` , `sys`	Rutas y ajuste de `sys.path` en scripts y pruebas	`interfaz/app.py` , `tests/*.py` , `experimentos/*.py`
`unittest`	Pruebas automatizadas	`tests/*.py`

4.3 Herramientas y convenciones

Herramienta / convención	Observación
Git	Repositorio con rama `main` ; sin etiquetas (`git tag` vacío)
`.gitignore`	Ignora `__pycache__/` , `*.py[cod]` , `*$py.class` , `.vscode/` , `.idea/`
Documentación de funciones	Docstrings con campos `Purpose` , `Preconditions` , `Postconditions` , `Complexity` , `AI usage` , `Student validation`
Empaquetado	No hay archivos `__init__.py` salvo `algoritmos/__init__.py` ; el resto de carpetas funciona como paquetes de espacio de nombres implícitos (Python 3.3+)
Linter / formateador / CI	No se encontró configuración de ninguno
Cobertura de pruebas	No se encontró configuración de cobertura

4.4 Directorios sin contenido

- `heuristicas/` : existe en el sistema de archivos, sin archivos.
- `docs/` : **ya no existe** en el sistema de archivos (constaba como vacío en la versión 1.0 del manual).

> Al estar vacío, Git no versiona `heuristicas/` . Su contenido es `[PENDIENTE DE IMPLEMENTACIÓN]` . La documentación del proyecto vive en `documentacion/` (manual técnico, manual de usuario y resultados experimentales).

5. Estructura de carpetas y responsabilidad de archivos

```
TacticalGrid/
├── main.py
├── README.md
├── .gitignore
├── algoritmos/
│   ├── __init__.py
│   ├── base.py
│   ├── busqueda_no_informada.py
│   ├── busqueda_con_costo.py
│   ├── a_estrella.py
│   ├── beam_search.py
│   └── minimax.py
├── escenario/
│   ├── mapa.py
│   ├── modelos.py
│   ├── cargador.py
│   └── validador.py
├── juego/
│   ├── estado.py
│   ├── reglas.py
│   └── problema_navegacion.py
├── interfaz/
│   ├── app.py
│   ├── tablero.py
│   └── graficas.py
├── verificacion/
│   └── verificador.py
├── documentacion/
│   ├── manual_tecnico.md
│   ├── manual_de_usuario.md
│   └── resultados_experimentos_busqueda.md
├── resultados/
│   ├── tabla_esperado_vs_obtenido.csv
│   └── tabla_esperado_vs_obtenido.md
├── tests/
│   ├── test_paso1_2.py
│   ├── test_paso3_ucs.py
│   ├── test_paso4_a_estrella.py
│   ├── test_beam_search.py
│   └── test_minimax.py
├── experimentos/
│   ├── generar_casos_minimos.py
│   ├── generar_casos_robustez.py
│   ├── caso06_estado_compuesto.py
│   ├── ejecutar_experimentos_busqueda.py
│   ├── ejecutar_casos_adversariales.py
│   ├── ejecutar_casos.py
│   └── esperados.json
├── escenarios/
│   ├── ejemplo.json
│   ├── prueba_busqueda_dfs.json
│   ├── prueba_partida.json
│   ├── prueba_adversarial.json
│   ├── invalido_a_proposito.json
│   ├── casos minimos/
│   │   ├── caso01_bfs_vs_dfs.json
│   │   ├── caso2_ruta_corta_vs_economica.json
│   │   ├── caso3_obstaculo_y_heuristica.json
│   │   ├── caso4_comparacion_heuristicas.json
│   │   ├── caso05_beam_search.json
│   │   ├── caso06a_sin_recurso.json
│   │   ├── caso06b_con_recurso.json
```

```
│   │   ├── caso07_decision_adversarial.json
│   │   ├── caso08_minimax_vs_alfa_beta.json
│   │   ├── caso09_orden_acciones.json
│   │   └── caso10_profundidad.json
│   ├── casos_invalidos/
│   │   ├── inv01_dimensiones_inconsistentes.json
│   │   ├── inv02_terreno_inexistente.json
│   │   ├── inv03_costo_cero.json
│   │   ├── inv04_base_fuera_del_mapa.json
│   │   ├── inv05_unidad_sobre_muro.json
│   │   ├── inv06_id_repetido.json
│   │   ├── inv07_bando_invalido.json
│   │   ├── inv08_turno_invalido.json
│   │   ├── inv09_portador_inexistente.json
│   │   ├── inv10_json_mal_formado.json
│   │   ├── inv11_falta_campo_obligatorio.json
│   │   └── inv12_recurso_fuera_del_mapa.json
│   └── casos_robustos/
│       ├── r01_meta_inalcanzable.json
│       ├── r02_inicio_igual_meta.json
│       ├── r03_corredor_1x20.json
│       ├── r04_unidad_encerrada.json
│       ├── r05_objetivo_en_muro.json
│       ├── r06_costos_fraccionarios.json
│       ├── r07_dimensiones_y_nombres_nuevos.json
│       ├── r08_mapa_grande_40x40.json
│       ├── r09_portador_en_su_base.json
│       └── r10_unidad_sobre_el_recurso.json
└── heuristicas/       (vacío)
```

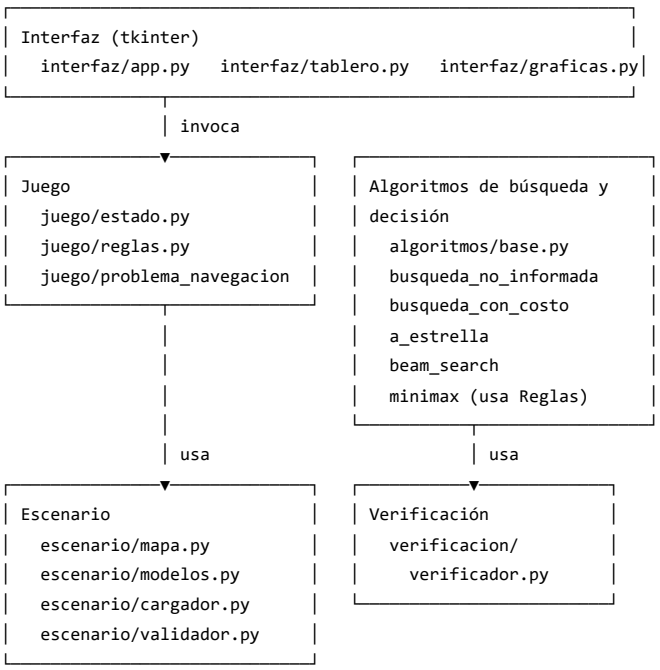
5.1 Responsabilidad por archivo

Archivo	Responsabilidad	Estado
`main.py`	Punto de entrada: instancia `App` y arranca `mainloop()`	Implementado
`README.md`	Instrucciones básicas de ejecución y modos	Implementado (integrantes sin completar)
`.gitignore`	Exclusiones de control de versiones	Implementado
`algoritmos/__init__.py`	Importa los módulos de algoritmos para activar su registro	Implementado
`algoritmos/base.py`	Contrato común, registro de algoritmos, métricas (`ResultadoBusqueda`), reconstrucción de caminos y medición de tiempo	Implementado
`algoritmos/busqueda_no_informada.py`	BFS y DFS	Implementados
`algoritmos/busqueda_con_costo.py`	UCS / Dijkstra	Implementado
`algoritmos/a_estrella.py`	A*, heurísticas `manhattan` , `propia` y `euclidiana` , selector de heurística	Implementado
`algoritmos/beam_search.py`	Beam Search (`BEAM_SEARCH`) con ancho de haz `k` y heurística configurable	Implementado
`algoritmos/minimax.py`	Minimax y poda alfa-beta sobre el estado compuesto, con `ResultadoAdversarial`	Implementado
`escenario/mapa.py`	Representación de la cuadrícula y consultas de terreno	Implementado
`escenario/modelos.py`	`Escenario` validado y construcción del estado inicial	Implementado
`escenario/cargador.py`	Lectura JSON, validación y construcción «todo o nada»	Implementado
`escenario/validador.py`	Reglas de validación del JSON	Implementado

Archivo	Responsabilidad	Estado
`juego/estado.py`	Entidades inmutables `Unidad`, `Estado`, `Accion`	Implementado
`juego/reglas.py`	Acciones legales, costo, transición, ganador y terminalidad	Implementado
`juego/problema_navegacion.py`	Formulación del problema de navegación de un agente	Implementado
`interfaz/app.py`	Ventana principal, orquestación de tareas, resultados, estadísticas y comparación	Implementado
`interfaz/tablero.py`	Widget de dibujo del tablero y paleta de colores	Implementado
`interfaz/graficas.py`	Gráfico de barras simple sobre un `Canvas`	Implementado
`verificacion/verificador.py`	Comprobaciones de validez de un camino	Implementado
`tests/*.py`	Pruebas unitarias con `unittest` (carga/reglas, UCS, A*, Beam Search y Minimax/alfa-beta)	Implementado
`experimentos/generar_casos_minimos.py`	Generación reproducible de los casos mínimos 1, 5 y 6	Implementado
`experimentos/generar_casos_robustez.py`	Generación reproducible del caso 2, de los 10 escenarios robustos y de los 12 inválidos	Implementado
`experimentos/caso06_estado_compuesto.py`	Demostración por consola del caso 6	Implementado
`experimentos/ejecutar_experimentos_busqueda.py`	Genera `documentacion/resultados_experimentos_busqueda.md` (casos 1–5, robustos R01–R08 y adversariales 7–10)	Implementado
`experimentos/ejecutar_casos_adversariales.py`	Reporte por consola de los casos 7, 8, 9 y 10	Implementado
`experimentos/esperados.json`	Declaración de valores esperados previos a la ejecución (C01–C10, R01–R10, V01–V12)	Implementado
`experimentos/ejecutar_casos.py`	Tabla esperado vs. obtenido a partir de `esperados.json`	**No ejecutable** (importa módulos inexistentes; véase 15.2)
`documentacion/*.md`	Manual técnico, manual de usuario y resultados experimentales	Implementado
`resultados/tabla_esperado_vs_obtenido.*`	Salida generada por `ejecutar_casos.py`	**Desactualizada** (todas las filas dicen `DIFIERE` por rutas inexistentes de una corrida anterior)
`escenarios/*.json`	Datos de escenario y pruebas declarativas	Implementado

6. Arquitectura del sistema

6.1 Vista de capas



6.2 Dependencias entre módulos (importaciones verificadas)

Módulo	Importa de
`main.py`	`interfaz.app`
`interfaz/app.py`	`algoritmos`, `algoritmos.base`, `algoritmos.minimax`, `escenario.cargador`, `interfaz.graficas`, `interfaz.tablero`, `juego.problema_navegacion`, `juego.reglas`, `verificacion.verificador`
`interfaz/graficas.py`	`interfaz.tablero`
`interfaz/tablero.py`	Solo `tkinter`
`algoritmos/busqueda_no_informada.py`, `busqueda_con_costo.py`, `a_estrella.py`, `base.py`	`algoritmos.base`
`algoritmos/beam_search.py`	`algoritmos.base` y `algoritmos.a_estrella` (`obtener_heuristica`)
`algoritmos/minimax.py`	Ninguno del proyecto (solo `dataclasses` y `time`); recibe `reglas` y el estado por parámetro
`escenario/cargador.py`	`escenario.mapa`, `escenario.modelos`, `escenario.validador`, `juego.estado`
`escenario/modelos.py`	`escenario.mapa`, `juego.estado`
`juego/reglas.py`	`escenario.mapa` (`DIRECCIONES`), `juego.estado`
`juego/problema_navegacion.py`	Ninguno
`verificacion/verificador.py`	Ninguno
`tests/*`	módulos del proyecto, ajustando `sys.path`

No se detectaron dependencias circulares a nivel de importación. `escenario.modelos` importa `juego.estado`, y `juego.estado` no importa nada del proyecto.

6.3 Flujo de ejecución principal

1. **Arranque**: `main.py` → `App()` → `_construir()` crea barra superior, tablero y pestañas → `_mostrar_frame_tarea()` → mensaje de estado inicial.
2. **Carga de escenario** (`App.cargar_dialogo` / `App.cargar_ruta`):
   - `cargar_escenario(ruta)` lee el JSON.
   - `construir_escenario(datos)` llama a `validar_datos(datos)` .
   - Si hay errores → `ErrorEscenario` con lista completa; la interfaz los muestra y **no reemplaza** el escenario anterior.
   - Si es válido → se construyen `Mapa` , `Unidad` y `Escenario` .
3. **Aplicación del escenario** (`App._aplicar_escenario`): se crean `Reglas` , el estado inicial, se ajustan controles y se llama a `cambiar_modo(auto_ejecutar=True)` .
4. **Modo JSON**: `_aplicar_prueba()` lee `prueba` y ajusta tarea, unidad, objetivo, algoritmo, heurística y `k` ; `_bloquear(True)` deshabilita controles. Si existe `prueba` , se ejecuta automáticamente.
5. **Modo exploración**: el usuario elige tarea, algoritmo, unidad y objetivo; los controles permanecen habilitados.
6. **Búsqueda**:
   - `_problema()` construye un `ProblemaNavegacion(mapa, inicio, objetivo)` .
   - `_correr()` invoca `ejecutar_algoritmo(nombre, problema, k=..., heuristica=...)` .
   - El algoritmo registrado se ejecuta y devuelve `ResultadoBusqueda` .
   - `verificar_camino(...)` produce las comprobaciones.
   - Se agrega un registro al historial y se muestran resultados y métricas.
   - Si el algoritmo es A*, además se ejecuta la comparación entre heurísticas.
7. **Comparación de algoritmos de búsqueda**: `comparar_todos()` ejecuta todos los algoritmos de búsqueda registrados (BFS, DFS, UCS, A*, Beam Search) sobre el mismo problema.
8. **Partida**: clics en el tablero → `Reglas.acciones_legales` / `Reglas.aplicar` ; los bandos automáticos («Azar», Minimax, Alfa-Beta) eligen su acción solos.
9. **Adversarial**: `_ejecutar_adversarial()` evalúa el estado compuesto con `_evaluar_y_aplicar_adversarial()` , que corre Minimax y Alfa-Beta en paralelo vía `_correr_adversarial()` , aplica la jugada elegida y la resalta en el tablero; `_mostrar_resultado_adversarial()` vuelca métricas y la comparación al panel.

6.4 Componentes y relaciones

Componente	Rol	Relación principal
`Mapa`	Cuadrícula y costos de terreno	Usado por `ProblemaNavegacion` , `Reglas` , `Escenario` , `Tablero`
`Escenario`	Datos validados del escenario	Produce `Estado` inicial; usado por `Reglas` y la interfaz
`Unidad` , `Estado` , `Accion`	Modelo inmutable del juego	Entradas/salidas de `Reglas`
`Reglas`	Legalidad, transición, costo, victoria	Orquesta el estado del juego
`ProblemaNavegacion`	Interfaz del problema para búsqueda	Consumida por los algoritmos de búsqueda (BFS, DFS, UCS, A*, Beam Search)
`ResultadoBusqueda`	Resultado y métricas de una búsqueda	Producida por algoritmos de búsqueda, consumida por interfaz y verificador
`ResultadoAdversarial`	Resultado y métricas de una decisión adversarial	Producida por `minimax()` / `alfa_beta()` , consumida por la interfaz y los experimentos
`minimax` / `alfa_beta`	Búsqueda adversarial sobre el estado compuesto	Reciben `Reglas` , estado inicial y profundidad; usan `Reglas.acciones_legales` / `aplicar`
Registro de algoritmos (`REGISTRO`)	Descubrimiento de algoritmos	Poblado por decorador `@registrar` al importar `algoritmos`
`Tablero`	Dibujo	Recibe escenario, estado y capas visuales
`verificar_camino`	Validación independiente	Recibe mapa, inicio, objetivo y resultado

7. Modelado del problema, representación de estados y modelo de datos

7.1 Formalización del Problema de Navegación Unipersonal (Búsqueda de Rutas)

El primer problema abordado en TacticalGrid corresponde a la **navegación de un agente individual** desde una posición inicial hasta un objetivo sobre una cuadrícula con costos heterogéneos y obstáculos. Siguiendo el estándar formal de Russell & Norvig (2020), este problema se modela mediante la tupla canónica de 6 elementos:

$$\mathcal{P}_{\text{nav}} = \langle S, s_0, A(s), T(s,a), G(s), c(s,a,s') \rangle$$

1. **Espacio de estados (S):**
   Conjunto de todas las posiciones o celdas transitables dentro de los límites del mapa:

   $$S = \{(f,c) \in \mathbb{N}_0^2 \mid 0 \leq f < F,\ 0 \leq c < C,\ \text{es_transitable}(f,c) = \text{True}\}$$

   donde F es el número de filas y C el número de columnas. Un estado individual $s \in S$ queda completamente determinado por la tupla de coordenadas (f, c).
2. **Estado inicial (s_0):**
   Posición inicial de la unidad asignada para la misión de búsqueda:

   $$s_0 = (f_{\text{inicio}}, c_{\text{inicio}}) \in S$$

3. **Conjunto de acciones aplicables ($A(s)$):**
   Movimientos ortogonales en las 4 direcciones cardinales (deterministas y con orden canónico fijo):

   $$\mathcal{D} = \{(\text{ARRIBA}, -1, 0), (\text{ABAJO}, 1, 0), (\text{IZQUIERDA}, 0, -1), (\text{DERECHA}, 0, 1)\}$$

   Una acción $a = (\text{nombre}, \Delta f, \Delta c)$ es legal en el estado $s = (f, c)$ si y solo si la celda resultante pertenece al espacio de estados transitables:

   $$A(s) = \{a \in \mathcal{D} \mid (f + \Delta f, c + \Delta c) \in S\}$$

4. **Modelo de transición ($T(s,a)$):**
   Función determinista que asigna un único estado sucesor s' al aplicar la acción a sobre s:

   $$T(s,a) = (f + \Delta f, c + \Delta c) = s'$$

5. **Test de objetivo ($G(s)$):**
   Predicado booleano que determina si el estado actual s coincide exactamente con la celda meta designada $g = (f_g, c_g)$:

   $$G(s) = \begin{cases} \text{True} & \text{si } s = (f_g, c_g) \\ \text{False} & \text{en caso contrario} \end{cases}$$

6. **Función de costo de paso ($c(s,a,s')$):**
   El costo de realizar la transición hacia s' está dado por el costo intrínseco del tipo de terreno de la celda de destino s':

   $$c(s,a,s') = \text{costo}(s') \quad \text{con } \text{costo}(s') \geq c_{\min} > 0$$

   El costo acumulado de una trayectoria $P = (s_0, s_1, \ldots, s_k)$ es aditivo:

   $$g(P) = \sum_{i=1}^{k} c(s_{i-1}, a_i, s_i) = \sum_{i=1}^{k} \text{costo}(s_i)$$

7.2 Formalización del Juego Táctico por Turnos (Espacio de Estados Compuesto)

El segundo problema modelado en TacticalGrid es una **escaramuza táctica por turnos para dos jugadores** (bando A vs. bando B) con captura y transporte de un recurso estratégico. Se formaliza mediante la tupla:

$$\mathcal{M}_{\text{juego}} = \langle S_{\text{juego}}, s_0, \mathcal{P}, A_{\text{juego}}(s), T_{\text{juego}}(s,a), \text{Terminal}(s), \text{Ganador}(s) \rangle$$

1. **Jugadores (Bandos):** $\mathcal{P} = \{\mathrm{A}, \mathrm{B}\}$.
2. **Espacio de estados compuesto (S_{juego}):**
   Un estado global $s \in S_{\text{juego}}$ es una 4-tupla inmutable y canónica:

$$s = (U, r, p, t)$$

   - $U = (u_1, u_2, \ldots, u_m)$**:** Tupla ordenada canónicamente por `id` con las unidades actualmente vivas en el tablero. Cada unidad es $u_i = (\mathrm{id}_i, \mathrm{bando}_i, f_i, c_i)$ donde $(f_i, c_i) \in S$.
   - $r \in S \cup \{\mathrm{None}\}$**:** Posición en el suelo del recurso neutral (f_r, c_r). Si una unidad porta el recurso, $r = \mathrm{None}$.
   - $p \in \{\mathrm{id}_1, \ldots, \mathrm{id}_m\} \cup \{\mathrm{None}\}$**:** Identificador de la unidad portadora del recurso. Si el recurso está libre en el suelo, $p = \mathrm{None}$.
   - $t \in \{\mathrm{A}, \mathrm{B}\}$**:** Bando que posee el turno activo.
3. **Invariantes del estado compuesto:**
   - **Exclusión de portador:** $p = \mathrm{None} \iff r \neq \mathrm{None}$. El recurso nunca puede estar simultáneamente en el suelo y en manos de un agente, ni desaparecer del juego.
   - **No superposición regular:** Dos unidades no pueden ocupar la misma coordenada en reposo:

$$\forall i \neq j, \quad (f_i, c_i) \neq (f_j, c_j)$$

   - **Excepción de interceptación:** Una unidad del bando activo puede entrar a una casilla ocupada si y solo si dicha casilla está ocupada por el portador del recurso del bando rival. Dicha acción desencadena la eliminación inmediata del portador rival y la liberación del recurso.
4. **Acciones del juego ($A_{\text{juego}}(s)$):**
   Corresponden a las acciones legales para las unidades pertenecientes al bando en turno t:
   - $\mathrm{MOVER}(u_i, f', c')$: Movimiento ortogonal a una casilla transitable adyacente no ocupada (o ocupada por el portador enemigo).
   - $\mathrm{ESPERAR}(u_i)$: La unidad permanece en su posición actual sin consumir costo ($c = 0$).
5. **Condición de victoria y terminalidad:**
   - $\mathrm{Terminal}(s) = \mathrm{True} \iff \mathrm{Ganador}(s) \in \{\mathrm{A}, \mathrm{B}\}$.
   - El bando $\mathrm{Bando} \in \{\mathrm{A}, \mathrm{B}\}$ gana si:
     a. Una unidad de su bando porta el recurso y se encuentra posicionada sobre su respectiva base:

$$p = u_i.\mathrm{id} \land u_i.\mathrm{bando} = \mathrm{Bando} \land (u_i.f, u_i.c) = \mathrm{base}[\mathrm{Bando}]$$

     b. O el bando rival no posee ninguna unidad viva restante sobre el tablero (victoria por aniquilación/interceptaciones).

7.3 Comparación de Espacios y Análisis Combinatorio ($|S|$)

Una distinción teórica y práctica fundamental en TacticalGrid radica en la diferencia de magnitud entre el **espacio de navegación individual** y el **espacio compuesto del juego**:

Propiedad	Espacio de Navegación Unipersonal	Espacio Compuesto del Juego Táctico
Estructura del estado	Coordenadas 2D: (f, c)	Tupla compuesta: (U, r, p, t)
Agentes	1 unidad aislada	Múltiples unidades de dos bandos rivales
Dinámica	Determinista y estática (terreno inalterable)	Determinista por turnos con captura, eliminación y alternancia
Objetivo	Alcanzar una celda fija g	Llevar el recurso a la base propia o eliminar al rival
Algoritmos aptos	BFS, DFS, UCS, A*, Beam Search	Minimax, Poda Alfa-Beta, Simulación Táctica

Estimación Cuantitativa del Tamaño del Espacio de Estados ($|S|$)

Sea $K = |S|$ el número de casillas transitables del mapa de dimensiones $F \times C$ ($K \leq F \cdot C$).

A. Tamaño del Espacio de Navegación ($|S_{\text{nav}}|$)

Para el problema de navegación, un estado es simplemente una casilla transitable:

$$|S_{\text{nav}}| = K \leq F \times C$$

- En un mapa de 20×20 con 10% de obstáculos:

$$|S_{\text{nav}}| \approx 360 \text{ estados}$$

Este tamaño reducido permite que algoritmos exhaustivos como UCS o informados como A* exploren el espacio en pocos milisegundos sin desbordar memoria.

B. Tamaño del Espacio Compuesto de la Partida ($|S_{\text{juego}}|$)

Para el juego completo, intervienen:

1. **Posición de las unidades:** Sean N_A unidades del bando A y N_B unidades del bando B, con $N = N_A + N_B$ unidades vivas en total. Dado que las unidades tienen identificadores únicos y no pueden compartir casilla simultáneamente, el número de configuraciones espaciales válidas es una permutación de K celdas tomadas de N en N:
$$P(K, N) = \frac{K!}{(K-N)!}$$
2. **Estado del recurso y portador:**
   - Si el recurso está libre en el suelo ($p = \text{None}$), puede ocupar cualquiera de las $K - N$ celdas que no tienen unidades encima ($K - N$ opciones).
   - Si el recurso lo porta una unidad ($r = \text{None}$), cualquiera de las N unidades puede ser la portadora (N opciones).
   - Por tanto, el número de estados posibles para el subsistema del recurso es:
$$(K - N) + N = K \text{ configuraciones}$$
3. **Turno de juego:** 2 estados posibles ($t \in \{\text{A}, \text{B}\}$).

Combinando todas las dimensiones independientes:

$$|S_{\text{juego}}| = 2 \times K \times P(K, N) = 2 \cdot K \cdot \frac{K!}{(K-N)!}$$

Demostración de la Explosión Combinatoria

Tomando los parámetros estándar de un escenario de TacticalGrid:

- Dimensiones: Cuadrícula de 20×20 con $K = 360$ celdas transitables.
- Unidades: 3 unidades por bando ($N_A = 3, N_B = 3 \implies N = 6$ unidades en total).

Calculamos el número de permutaciones de las unidades:

$$P(360, 6) = 360 \times 359 \times 358 \times 357 \times 356 \times 355 \approx 2.147 \times 10^{15}$$

Multiplicando por las $K = 360$ variantes del recurso y los 2 turnos:

$$|S_{\text{juego}}| = 2 \times 360 \times (2.147 \times 10^{15}) \approx 1.546 \times 10^{18} \text{ estados}$$

> **Conclusión del análisis de complejidad espacial:**
> Mientras que el problema de navegación individual posee un orden de magnitud de **centenas de estados** (10^2), el juego táctico por turnos alcanza un orden de magnitud superior a **un trillón y medio de estados** (1.5×10^{18}).
> Esta disparidad matemática justifica formalmente por qué es imposible resolver la partida mediante búsqueda exhaustiva hacia adelante (como BFS o UCS) y se vuelve indispensable el uso de **búsqueda adversarial con cota de profundidad (Minimax), técnicas de poda (Alfa-Beta) y funciones de evaluación heurística**.

7.4 Entidades internas (código)

7.4.1 `Mapa` — `escenario/mapa.py`

`@dataclass(frozen=True)`

Atributo	Tipo declarado (anotación)	Descripción verificable
`filas`	`int` (por uso)	Número de filas

Atributo	Tipo declarado (anotación)	Descripción verificable
`columnas`	`int` (por uso)	Número de columnas
`terreno`	`tuple`	Tupla de tuplas con nombres de tipo de terreno
`tipos`	`dict`	`nombre → {"transitable": bool, "costo": número o None}`

Observación: aunque la clase es `frozen`, contiene un `dict` (`tipos`), por lo que no es hashable. En el código no se usa como clave de diccionario ni en conjuntos.

Constante del módulo:

```
DIRECCIONES = (("ARRIBA", -1, 0), ("ABAJO", 1, 0), ("IZQUIERDA", 0, -1), ("DERECHA", 0, 1))
```

El orden es determinista y afecta el orden de generación de sucesores.

7.4.2 `Unidad` — `juego/estado.py`

`@dataclass(frozen=True)`

Atributo	Tipo	Descripción
`id`	`str`	Identificador único dentro del escenario
`bando`	`str`	`"A"` o `"B"`
`fila`	`int`	Fila actual
`columna`	`int`	Columna actual

El campo `tipo` presente en el JSON **no se traslada** a `Unidad` (véase `escenario/cargador.py`, línea 34).

7.4.3 `Estado` — `juego/estado.py`

`@dataclass(frozen=True)`

Atributo	Tipo	Descripción
`unidades`	`tuple[Unidad]`	Ordenada por `id` (forma canónica)
`recurso_pos`	`Optional[tuple]`	`(f, c)` si el recurso está en el suelo; `None` si alguien lo lleva
`portador`	`Optional[str]`	`id` de la unidad que lleva el recurso, o `None`
`turno`	`str`	Bando que debe actuar (`"A"` o `"B"`)

Al ser `frozen` y contener solo tuplas y cadenas, es hashable. La prueba `test_hashable_e_inmutable` lo comprueba.

7.4.4 `Accion` — `juego/estado.py`

`@dataclass(frozen=True)`

Atributo	Tipo	Descripción
`unidad_id`	`str`	Unidad que actúa
`tipo`	`str`	`"MOVER"` o `"ESPERAR"`
`destino`	`Optional[tuple]`	`(f, c)` para `"MOVER"`; valor por defecto `None`

7.4.5 `Escenario` — `escenario/modelos.py`

`@dataclass(frozen=True)`

Atributo	Tipo	Descripción
`mapa`	`Mapa`	Cuadrícula
`bases`	`dict`	`{"A": (f,c), "B": (f,c)}`
`recurso`	`tuple`	Posición inicial del recurso
`unidades`	`tuple`	Tupla de `Unidad`
`turno`	`str`	Bando inicial
`portador`	`Optional[str]`	Portador inicial del recurso
`profundidad_minimax`	`int`	Profundidad máxima declarada
`prueba`	`Optional[dict]`	Bloque `prueba` del JSON, sin transformar

7.4.6 `ResultadoBusqueda` — `algoritmos/base.py`

`@dataclass` (mutable; no `frozen`)

Atributo	Tipo / valor por defecto	Descripción
`algoritmo`	`str`	Nombre del algoritmo
`exito`	`bool`	Si encontró solución
`camino`	`list` (`[]`)	Lista de `(fila, columna)`
`costo`	`Optional[float]`	Costo acumulado o `None`
`estados_generados`	`int` (`0`)	Estados insertados en la frontera
`estados_expandidos`	`int` (`0`)	Estados cuyos sucesores se analizaron
`maximo_frontera`	`int` (`0`)	Tamaño máximo observado de la frontera
`tiempo_ms`	`float` (`0.0`)	Tiempo medido por `ejecutar_algoritmo`
`explorados`	`list` (`[]`)	Estados expandidos en orden
`parametros`	`dict` (`{}`)	Parámetros con los que se ejecutó

Propiedad `longitud` : `len(camino) - 1` si `exito` y hay camino; en caso contrario `None` .

7.4.7 `ResultadoAdversarial` — `algoritmos/minimax.py`

`@dataclass` (mutable)

Atributo	Tipo	Descripción
`algoritmo`	`str`	`"MINIMAX"` o `"ALFA_BETA"`
`accion`	`Accion` o `None`	Mejor acción hallada desde la perspectiva inicial
`valor`	`float`	Utilidad minimax del estado raíz para esa perspectiva
`profundidad_maxima`	`int`	Nivel de *plies* explorado
`nodos_generados`	`int`	Estados hijos creados durante la exploración
`nodos_evaluados`	`int`	Hojas/estados sin acciones cuya utilidad se calculó
`nodos_podados`	`int`	Ramas hermanas descartadas por el corte $\alpha \geq \beta$ (0 en Minimax puro)
`tiempo_ms`	`float`	Duración medida con `perf_counter`
`orden_acciones`	`str`	`estandar` , `inverso` o `heuristico`

Método `a_dict()` : serializa a `dict` (acción reducida a `unidad_id` , `tipo` y `destino` ; `None` si no hay acción) para el panel de resultados y los experimentos.

7.5 Formato JSON del escenario

Estructura observada en `escenarios/ejemplo.json`:

Clave	Tipo JSON	Obligatoria	Validación verificada
`version`	cadena	Sí	Solo se exige su presencia; el valor no se compara
`mapa.filas`	entero	Sí	Entero positivo; no booleano
`mapa.columnas`	entero	Sí	Entero positivo; no booleano
`tipos_terreno`	objeto	Sí	No vacío; cada tipo debe tener `transitable` booleano; si es transitable, `costo` numérico > 0
`terreno`	matriz	Sí	Lista de listas con dimensiones coherentes; cada nombre debe existir en `tipos_terreno`
`bases.A`, `bases.B`	objeto `{fila, columna}`	Sí	Enteros y dentro del mapa
`recurso`	objeto `{fila, columna}`	Sí	Enteros y dentro del mapa
`unidades`	lista	Sí	No vacía; cada elemento con `id`; ids únicos; `bando` A/B; posición entera, dentro del mapa y transitable
`turno`	cadena	Sí	A/B (se normaliza a mayúsculas)
`juego.portador_recurso`	cadena o `null`	Sí	Si no es `null`, debe corresponder a un `id` existente
`juego.profundidad_maxima_minimax`	entero	No	Valor por defecto: `3` (`PROFUNDIDAD_POR_DEFECTO`)
`prueba`	objeto	No	No se valida al cargar; la interfaz valida `modo` al ejecutar

Campos opcionales dentro de `prueba` (leídos por `App._aplicar_prueba`): `modo`, `unidad_inicio`, `objetivo{fila,columna}`, `algoritmo`, `heuristica`, `k`.

Observaciones verificables:

- `recurso` y `bases` se validan dentro del mapa, pero **no** se valida que estén sobre terreno transitable.
- El campo `tipo` de cada unidad se acepta en el JSON pero no se modela internamente.
- `version` se exige como clave, pero su contenido no se valida.
- `prueba.modo` debe ser `busqueda`, `partida` o `adversarial` para que `App.ejecutar` no la rechace.

7.6 Relaciones

```
Escenario 1 ── 1 Mapa
Escenario 1 ── * Unidad        (tupla ordenada)
Escenario 1 ── 1 Estado        (mediante estado_inicial)
Estado    1 ── * Unidad        (tupla ordenada por id)
Estado    1 ── 0..1 portador   (referencia a Unidad.id)
Reglas    1 ── 1 Escenario
ProblemaNavegacion 1 ── 1 Mapa
ResultadoBusqueda 1 ── * celdas (camino)
```

8. Descripción técnica de clases, funciones y métodos

Convención de estado: **Implementado** = funcionalidad conectada y con lógica verificable en el código; **Parcial** = existe parte de la funcionalidad; **Pendiente** = declarada o prevista pero sin implementación; **Informativo** = constante, dato o utilidad.

8.1 `main.py`

Elemento	Línea	Responsabilidad	Estado
Bloque `if __name__ == "__main__"`	4	Crea `App()` y llama a `mainloop()`	Implementado

8.2 `algoritmos/base.py`

Elemento	Línea	Responsabilidad	Estado
Constante `PLANIFICADOS`	12	Tuple vacía `()` (todos los algoritmos previstos ya están implementados); su comentario documenta `BFS, DFS, UCS, A*, Beam Search`	Informativo (sin uso pendiente)
`REGISTRO`	14	Diccionario global `nombre → función`	Implementado
`ResultadoBusqueda`	18	Contenedor de resultados y métricas	Implementado
`ResultadoBusqueda.longitud`	32	Propiedad calculada del número de movimientos	Implementado
`ResultadoBusqueda.a_dict`	36	Serialización a diccionario con tiempo redondeado a 4 decimales	Implementado
`registrar(nombre)`	46	Decorador que inserta la función en `REGISTRO`	Implementado
`disponibles()`	54	Lista de nombres registrados en orden de inserción	Implementado
`normalizar_nombre(texto)`	59	Normaliza `a*` → `A_ESTRELLA`, espacios/guiones → `_`	Implementado
`reconstruir_camino(padre, meta)`	65	Reconstruye el camino y suma costos siguiendo `padre`	Implementado
`ejecutar_algoritmo(nombre, problema, **params)`	84	Invoca el algoritmo registrado, mide tiempo y adjunta parámetros (`k`, `heuristica`)	Implementado

Orden de registro efectivo al importar `algoritmos`: `BFS`, `DFS`, `UCS`, `A_ESTRELLA`, `BEAM_SEARCH` (determinado por el orden de importación en `algoritmos/__init__.py` y por el orden de las funciones decoradas).

8.3 `algoritmos/busqueda_no_informada.py`

Elemento	Línea	Responsabilidad	Estado
`_exito(...)`	7	Construye un `ResultadoBusqueda` exitoso reconstruyendo el camino	Implementado
`bfs(problema, **params)`	13	Búsqueda en anchura; marca visitados al generar	Implementado
`dfs(problema, **params)`	42	Búsqueda en profundidad con pila; marca visitados al expandir	Implementado

Observación verificable: BFS y DFS **no** usan los costos para elegir el camino, pero `reconstruir_camino` sí suma los costos del camino hallado, por lo que `ResultadoBusqueda.costo` refleja el costo del camino encontrado, no necesariamente el mínimo.

8.4 `algoritmos/busqueda_con_costo.py`

Elemento	Línea	Responsabilidad	Estado
`_exito(...)`	7	Construye resultado exitoso	Implementado
`ucs(problema, **params)`	13	Búsqueda de costo uniforme con `heapq` y diccionario `mejor_g`	Implementado

8.5 `algoritmos/a_estrella.py` y Demostración Formal de Heurísticas

Elemento	Línea	Responsabilidad	Estado
`_exito(...)`	8	Construye el contenedor de resultado exitoso	Implementado
`h_manhattan(s, problema)`	13	Distancia Manhattan escalada por $c_{\min}$	Implementado

Elemento	Línea	Responsabilidad	Estado
`h_propia(s, problema)`	25	Heurística informada por el costo del terreno de la meta	Implementado
`h_euclidiana(s, problema)`	40	Distancia Euclidiana escalada por $c_{\min}$	Implementado
`HEURISTICAS`	52	Diccionario `{"manhattan": ..., "propia": ..., "euclidiana": ...}`	Implementado
`obtener_heuristica(nombre)`	59	Devuelve la función según la clave (por defecto `manhattan`)	Implementado
`a_estrella(problema, **params)`	64	A* con cola de prioridad, desempate por `h` y contador	Implementado

Observaciones técnicas del algoritmo:

- La función de evaluación es $f(s) = g(s) + h(s)$.
- La cola de prioridad (`heapq`) almacena tuplas de 4 elementos: `(f, h, contador, estado)` , permitiendo desempatar por menor valor heurístico $h(s)$ y en última instancia por orden de inserción (`contador`) para preservar determinismo estricto.
- Se mantiene el diccionario `mejor_g` que almacena el menor costo acumulado conocido hasta cada estado, evitando reinsertar caminos subóptimos.
- Se mantiene el conjunto `expandidos_set` : al extraer un nodo de la frontera, si ya fue expandido previamente, se descarta inmediatamente.
- La prueba de objetivo se realiza al momento de **extraer** el nodo de la frontera (garantía formal de optimalidad para A*).

8.5.1 Definición Matemática de las Heurísticas

Sea la cuadrícula definida sobre coordenadas enteras $\mathbb{Z}^2$, donde el estado actual es $s = (f, c)$ y el objetivo es $g = (f_g, c_g)$.
Definimos el costo mínimo entre todos los terrenos transitables del escenario como:

$$c_{\min} = \min\{\text{costo}(u, v) \mid (u, v) \in \text{mapa transitable}\}$$

Dado que las reglas de validación exigen que todo terreno transitable tenga un costo estrictamente positivo, se tiene la propiedad fundamental $c_{\min} > 0$.

1. **Distancia Manhattan L_1:**

   $$d_M(s, g) = |f - f_g| + |c - c_g|$$

   Representa el número mínimo absoluto de transiciones ortogonales requeridas para ir de s a g en una cuadrícula sin obstáculos.
2. **Heurística Manhattan escalada (h_M):**

   $$h_M(s) = d_M(s, g) \cdot c_{\min}$$

3. **Heurística Propia — Informada por el Terreno Objetivo (h_P):**

   $$h_P(s) = \begin{cases} 0 & \text{si } s = g \\ (d_M(s, g) - 1) \cdot c_{\min} + \text{costo}(g) & \text{si } s \neq g \end{cases}$$

   Intuición de diseño: Todo camino que parte de $s \neq g$ y llega a g debe realizar al menos $d_M(s, g)$ pasos. El último paso obligatoriamente ingresa a la celda objetivo g, por lo que su costo real es exactamente $\text{costo}(g)$. Los $d_M(s, g) - 1$ pasos anteriores deben costar al menos $c_{\min}$ cada uno.
4. **Heurística Euclidiana escalada (h_E):**

   $$h_E(s) = \sqrt{(f - f_g)^2 + (c - c_g)^2} \cdot c_{\min}$$

8.5.2 Demostración Formal de Admisibilidad

Una función heurística $h(s)$ es **admisible** si nunca sobreestima el costo real óptimo $h^*(s)$ para alcanzar la meta desde s:

$$\forall s, \quad 0 \leq h(s) \leq h^*(s)$$

A. Admisibilidad de $h_M(s)$ (Manhattan Escalada)

- **Paso 1 (Límite de pasos):** En una cuadrícula con movimiento restringido a 4 direcciones ortogonales, cada paso modifica la coordenada de fila o de columna en a lo sumo 1 unidad. En consecuencia, cualquier camino admisible $P = (s = v_0, v_1, \ldots, v_k = g)$ debe constar de una cantidad de pasos k tal que:

$$k \geq d_M(s, g)$$

  La presencia de obstáculos (muros) solo puede obligar al camino a desviarse, incrementando k, por lo que $k \geq d_M(s, g)$ se mantiene como una cota inferior estricta en el número de pasos.
- **Paso 2 (Costo por paso):** El costo total de cualquier camino P es la suma de los costos de entrar a cada celda sucesiva:

$$\text{costo}(P) = \sum_{i=1}^{k} \text{costo}(v_i)$$

  Dado que cada celda v_i es transitable, por definición de $c_{\min}$:

$$\text{costo}(v_i) \geq c_{\min} \quad \forall i \in \{1, \ldots, k\}$$

- **Paso 3 (Acotación del costo):** Sustituyendo la cota inferior de costo:

$$\text{costo}(P) = \sum_{i=1}^{k} \text{costo}(v_i) \geq \sum_{i=1}^{k} c_{\min} = k \cdot c_{\min} \geq d_M(s, g) \cdot c_{\min} = h_M(s)$$

- **Conclusión:** Puesto que la desigualdad $\text{costo}(P) \geq h_M(s)$ se cumple para todo camino válido, en particular se cumple para el camino de costo óptimo P^*, cuyo costo es $h^*(s)$:

$$h^*(s) = \text{costo}(P^*) \geq h_M(s) \implies h_M(s) \leq h^*(s)$$

  Por tanto, $h_M(s)$ **es estrictamente admisible**. ■

B. Admisibilidad de $h_P(s)$ (Heurística Propia)

- **Caso 1 ($s = g$):** $h_P(g) = 0$. Dado que el costo de estar en la meta y alcanzarla es 0, $h^*(g) = 0$, cumpliéndose $h_P(g) = h^*(g) = 0$.
- **Caso 2 ($s \neq g$):** Sea $P = (s = v_0, v_1, \ldots, v_{k-1}, v_k = g)$ cualquier camino válido de k pasos ($k \geq d_M(s, g) \geq 1$).
  Descomponemos el costo total aislando el último paso que entra a la meta $v_k = g$:

$$\text{costo}(P) = \left(\sum_{i=1}^{k-1} \text{costo}(v_i) \right) + \text{costo}(g)$$

  Para las $k - 1$ celdas intermedias $v_1, \ldots, v_{k-1}$, cada una satisface $\text{costo}(v_i) \geq c_{\min}$. Por ende:

$$\sum_{i=1}^{k-1} \text{costo}(v_i) \geq (k - 1) \cdot c_{\min}$$

  Como $k \geq d_M(s, g)$ y $c_{\min} > 0$:

$$(k - 1) \cdot c_{\min} \geq (d_M(s, g) - 1) \cdot c_{\min}$$

  Sumando el término $\text{costo}(g)$ en ambos lados:

$$\text{costo}(P) \geq (d_M(s, g) - 1) \cdot c_{\min} + \text{costo}(g) = h_P(s)$$

- **Conclusión:** Dado que esto es válido para cualquier camino, se cumple para el camino óptimo:

$$h^*(s) = \text{costo}(P^*) \geq h_P(s) \implies h_P(s) \leq h^*(s)$$

  Por tanto, $h_P(s)$ **es estrictamente admisible**. ■

8.5.3 Demostración Formal de Consistencia (Monotonicidad)

Una heurística $h(s)$ es **consistente** (o **monótona**) si para todo estado s y todo sucesor s' generado mediante una acción legal a, satisface la desigualdad triangular:

$$h(s) \leq c(s, a, s') + h(s') \quad \text{con} \quad h(g) = 0$$

donde $c(s, a, s') = \text{costo}(s')$ es el costo de transición al moverse a la celda s'.

A. Consistencia de $h_M(s)$ (Manhattan Escalada)

1. Sean s y s' dos celdas adyacentes conectadas por un paso ortogonal. Por la desigualdad triangular de la métrica L_1:
$$d_M(s, g) \leq d_M(s, s') + d_M(s', g)$$
2. Dado que s' es vecino ortogonal inmediato de s, la distancia Manhattan entre ellos es exactamente 1: $d_M(s, s') = 1$.
$$d_M(s, g) \leq 1 + d_M(s', g)$$
3. Multiplicando toda la desigualdad por el escalar positivo $c_{\min} > 0$:
$$d_M(s, g) \cdot c_{\min} \leq c_{\min} + d_M(s', g) \cdot c_{\min}$$
$$h_M(s) \leq c_{\min} + h_M(s')$$
4. Por definición de costo mínimo, el costo de ingresar a cualquier celda transitable s' cumple $\text{costo}(s') \geq c_{\min}$. En consecuencia:
$$c_{\min} \leq \text{costo}(s') = c(s, a, s')$$
5. Sustituyendo esta cota:
$$h_M(s) \leq c(s, a, s') + h_M(s')$$

   Demostrando que $h_M(s)$ **es consistente**. ■

B. Consistencia de $h_P(s)$ (Heurística Propia)

Analizamos los dos casos posibles para la transición $s \to s'$:

- **Caso 1: El sucesor es la meta ($s' = g$):**
  Como $s' = g$, tenemos $h_P(g) = 0$. La distancia Manhattan es $d_M(s, g) = 1$ porque s es vecino inmediato de la meta.
  Evaluamos $h_P(s)$:
  $$h_P(s) = (1 - 1) \cdot c_{\min} + \text{costo}(g) = \text{costo}(g)$$
  Evaluamos el lado derecho de la desigualdad:
  $$c(s, a, g) + h_P(g) = \text{costo}(g) + 0 = \text{costo}(g)$$
  Se cumple con igualdad exacta: $h_P(s) = c(s, a, g) + h_P(g)$.
- **Caso 2: El sucesor no es la meta ($s' \neq g$ con $s \neq g$):**
  Las expresiones de la heurística son:
  $$h_P(s) = (d_M(s, g) - 1) \cdot c_{\min} + \text{costo}(g)$$
  $$h_P(s') = (d_M(s', g) - 1) \cdot c_{\min} + \text{costo}(g)$$
  Restando ambas ecuaciones:

$$h_P(s) - h_P(s') = (d_M(s, g) - d_M(s', g)) \cdot c_{\min}$$

Por la propiedad de adyacencia ortogonal en la métrica Manhattan:

$$d_M(s, g) - d_M(s', g) \leq d_M(s, s') = 1$$

Por consiguiente:

$$h_P(s) - h_P(s') \leq 1 \cdot c_{\min} = c_{\min} \leq \text{costo}(s') = c(s, a, s')$$

$$h_P(s) \leq c(s, a, s') + h_P(s')$$

- **Conclusión:** Ambos casos satisfacen la condición monótona. Por tanto, $h_P(s)$ **es consistente**. ■

Importancia Teórica de la Consistencia en A*

La propiedad de consistencia garantiza que los valores evaluados de $f(n) = g(n) + h(n)$ a lo largo de cualquier camino son **monótonamente no decrecientes**.
Como corolario fundamental (Russell & Norvig, 2020), cuando A* expande un nodo s, **el costo $g(s)$ acumulado hasta ese momento es ya el óptimo global**:

$$g(s) = g^*(s)$$

Esto implica formalmente que:

1. **No se requiere reabrir estados cerrados:** Ningún estado una vez expandido puede volver a ser alcanzado por un camino más económico.
2. **Eficiencia en memoria:** Basta con marcar los nodos en un conjunto cerrado (`expandidos_set`) y descartar futuras apariciones, manteniendo la complejidad temporal y espacial óptima.

8.5.4 Análisis y Demostración Formal de Dominancia

Dadas dos heurísticas admisibles h_1 y h_2, se dice que h_2 **domina a** h_1 (denotado $h_2 \geq h_1$) si:

$$\forall s, \quad h_2(s) \geq h_1(s)$$

Teorema de Dominancia entre Heurística Propia y Manhattan:

$$\forall s, \quad h_P(s) \geq h_M(s)$$

- **Demostración:**
  - Si $s = g$: $h_P(g) = 0$ y $h_M(g) = 0 \cdot c_{\min} = 0$. Se cumple con igualdad: $h_P(g) = h_M(g)$.
  - Si $s \neq g$: Expresamos $h_P(s)$ algebraicamente en función de $h_M(s)$:

$$h_P(s) = (d_M(s, g) - 1) \cdot c_{\min} + \text{costo}(g) = d_M(s, g) \cdot c_{\min} - c_{\min} + \text{costo}(g)$$

$$h_P(s) = h_M(s) + (\text{costo}(g) - c_{\min})$$

Dado que g es una celda transitable del mapa, por definición de $c_{\min}$ se tiene:

$$\text{costo}(g) \geq c_{\min} \implies \text{costo}(g) - c_{\min} \geq 0$$

Sumando esta cantidad no negativa a $h_M(s)$:

$$h_P(s) = h_M(s) + (\text{costo}(g) - c_{\min}) \geq h_M(s)$$

  - **Desigualdad estricta:** Siempre que el objetivo esté situado en una celda de terreno con costo mayor que el costo mínimo global (por ejemplo en pasto $c = 2$, bosque $c = 4$ o pantano $c = 7$, donde $c_{\min} = 1$):

$$\text{costo}(g) > c_{\min} \implies h_P(s) > h_M(s) \quad \forall s \neq g$$

Por tanto, h_P **domina formalmente a** h_M. ■

Teorema de Poda en A* (Russell & Norvig, Cap. 3)

Si h_2 domina a h_1 y ambas son consistentes, entonces el conjunto de estados expandidos por A* con h_2 es un subconjunto del conjunto de estados expandidos por A* con h_1 (salvo posibles desempates sobre el umbral $f = C^*$):

$$\text{Expandidos}(A^*_{h_2}) \subseteq \text{Expandidos}(A^*_{h_1})$$

Justificación matemática:
A* expande necesariamente todos los estados que satisfacen $f(s) < C^*$, donde C^* es el costo óptimo de la solución.
Como $h_2(s) \geq h_1(s)$, se tiene:

$$f_2(s) = g(s) + h_2(s) \geq g(s) + h_1(s) = f_1(s)$$

Cualquier estado s para el cual $f_2(s) > C^*$ será podado de la expansión por $A^*_{h_2}$, mientras que ese mismo estado podría tener $f_1(s) < C^*$ y verse obligado a ser expandido por $A^*_{h_1}$.

Validación Empírica en TacticalGrid

En el experimento cuantitativo del **Caso 4** (`caso4_comparacion_heuristicas.json`), donde el objetivo g está ubicado en una celda de pantano ($c = 7 > c_{\min} = 1$):

- *A con Heurística Manhattan:** Expandió **353 estados**.
- *A con Heurística Propia:** Expandió **232 estados**.
- **Ahorro observado:** Se evitaron **121 expansiones de nodos** (una reducción del **34.3%** en el esfuerzo de búsqueda y del **27.6%** en tiempo de CPU: 2.218 ms frente a 3.062 ms en la corrida del 2026-10-20) encontrando idéntico camino de costo óptimo $C^* = 52.0$.

8.6 `algoritmos/beam_search.py` — Búsqueda en Haz con memoria acotada `k`

Elemento	Línea	Responsabilidad	Estado
`_exito(nombre, padre, meta, gen, exp, mf, explorados)`	6	Construye `ResultadoBusqueda` exitoso reconstruyendo el camino	Implementado
`beam_search(problema, **params)`	21	Haz de anchura `k` ; expande solo los `k` mejores estados de cada nivel, ordenados por heurística	Implementado (`@registrar("BEAM_SEARCH")`)

Comportamiento verificado:

- **Parámetros:** `k` (por defecto `2` , forzado a `max(1, int(k))` ; texto no numérico → `2`) y `heuristica` (`manhattan` por defecto, las mismas claves que A* vía `obtener_heuristica`).
- **Criterio de éxito:** la prueba de objetivo se hace **al inicio de cada nivel**, sobre los estados ya en el haz; los sucesores que no alcanzan el haz no se comprueban. Si el haz se vacía (sin candidatos) se termina con `exito = False` y `camino = []` .
- **Selección:** los candidatos del nivel se ordenan por tupla `(h(s), s)` , por lo que el desempate es determinista por orden lexicográfico del estado; se descartan duplicados del mismo nivel con `vistos_nivel` .
- **Métricas:** `estados_generados` cuenta los sucesores generados (padres registrados en `padre`), `estados_expandidos` los estados del haz realmente expandidos y `maximo_frontera` el mayor tamaño alcanzado por el haz (≤ `k`).

Resultados del Caso 5 (`caso05_beam_search.json`), re-ejecutados el 2026-10-20 (5 repeticiones, tiempo promedio):

k	¿Encuentra ruta?	Longitud	Costo	Generados	Expandidos	Máx. frontera	Tiempo (ms)
1	No	—	—	25	9	1	0.206
2	No	—	—	31	17	2	0.286
4	Sí	55	55.0	266	213	4	3.037
8	Sí	31 (óptimo)	31.0	276	234	8	2.906

(Líneas base del mismo escenario: BFS y A* encuentran el óptimo de 31 movimientos / costo 31.0; la verificación da `OK` para `k=4` y `k=8`.)

> **Nota de coherencia experimental:** la versión 1.0 del manual declaraba que `k=2` *sí* hallaba ruta. La re-ejecución muestra que `k=1` y `k=2` fracasan en este mapa, que `k=4` sortea el obstáculo con una solución **subóptima** (55 en vez de 31) y que `k=8` recupera el óptimo. La prueba `test_beam_search.py` se ajustó a este comportamiento medido (ver §11.1).

Observación teórica: con `h` admisible, **Beam Search no garantiza optimalidad** (podó estados en niveles anteriores que podrían contener el camino óptimo); el costo reportado es el del mejor camino encontrado, no necesariamente C^*. Su ventaja es el uso acotado de memoria: $O(k \cdot d)$ frente a $O(b^d)$ de DFS.

8.7 `algoritmos/minimax.py` — Minimax y poda Alfa-Beta

Elemento	Línea	Responsabilidad	Estado
`ResultadoAdversarial`	11	`dataclass` con acción, valor, profundidad, nodos generados/evaluados/podados, tiempo y orden	Implementado
`ResultadoAdversarial.a_dict()`	22	Serialización (acción → `unidad_id` / `tipo` / `destino`)	Implementado
`_distancia(a, b)`	40	Distancia Manhattan entre celdas (0 si alguno es `None`)	Implementado
`_utilidad(estado, reglas, escenario, perspectiva)`	46	Función de evaluación heurística (ver más abajo)	Implementado
`_clave_accion(accion)`	75	Clave de desempate `(unidad_id, tipo, destino)` para orden determinista	Implementado
`_acciones_ordenadas(...)`	79	Ordena las acciones legales según `orden_acciones`: `estandar`, `inverso`, `heuristico`	Implementado
`_buscar(...)`	92	Núcleo recursivo con o sin poda; contadores y cronómetro	Implementado
`minimax(estado, reglas, escenario, profundidad_maxima=3, orden_acciones="estandar")`	135	Búsqueda adversarial sin poda (`con_poda=False`)	Implementado
`alfa_beta(...)`	145	Misma búsqueda con poda alfa-beta (`con_poda=True`)	Implementado

Semántica. Ambas funciones comparten `_buscar`: la utilidad se evalúa **desde la perspectiva del bando que inicia la búsqueda** (`perspectiva = estado.turno`), la profundidad cuenta *plies* (acciones individuales, no rondas) y el nodo maximizador es aquel cuyo `s.turno` coincide con `perspectiva`.

Función de evaluación (`_utilidad`, línea 46):

1. **Nodos terminales:** `reglas.ganador` distinto de `None` $\rightarrow +100000$ si gana la perspectiva, -100000 en caso contrario (dominan cualquier valor heurístico).
2. **Material:** $35 \cdot (|\text{propios}| - |\text{enemigos}|)$.
3. **Recurso en juego (portador no nulo):** $\pm(180 - 8 \cdot d(\text{portador}, \text{base}))$ con signo según el bando del portador, más un bono/penalización de $\max(0, 5 - d_{\text{rival}}) \cdot 12$ por presión cercana sobre el portador.
4. **Recurso en el suelo:** $5 \cdot (d_{\text{rival al recurso}} - d_{\text{propio al recurso}})$ (premia acercar a las unidades propias).

Órdenes de acción (`_acciones_ordenadas`, línea 79): `estandar` = orden de `Reglas.acciones_legales`; `inverso` = su reversa; `heuristico` = evalúa una vez la utilidad de cada sucesor y ordena de mejor a peor para la perspectiva (desempate por `_clave_accion`). El orden **no cambia el valor óptimo**, solo el esfuerzo.

Poda alfa-beta: en la línea 123 se comprueba `alpha >= beta`; al podar se acumula `podados += len(acciones) - i - 1` (acciones hermanas no visitadas) y se corta con `break`. Con `con_poda=False` (Minimax puro) `nodos_podados` queda en `0`.

Validación de entrada (líneas 93–96): `profundidad_maxima` debe ser `int ≥ 1` y `orden_acciones` debe pertenecer a `("estandar", "inverso", "heuristico")`; en otro caso se lanza `ValueError`.

Complejidad: Minimax $O(b^d)$ en tiempo y $O(d)$ en espacio (recursión); alfa-beta $O(b^d)$ en el peor caso y hasta $O(b^{d/2})$ con orden ideal.

Equivalencia empírica comprobada por `tests/test_minimax.py::test_caso8_minimax_y_alfa_beta_equivalentes`: mismo valor y misma acción con profundidad 3, con reducción del esfuerzo (p. ej. Caso 8: de 3 255 a 543 nodos evaluados y 600 nodos podados; véase §13.8).

8.8 `escenario/mapa.py`

Elemento	Línea	Responsabilidad	Estado
`DIRECCIONES`	6	Orden fijo de movimientos: ARRIBA, ABAJO, IZQUIERDA, DERECHA	Informativo
`Mapa`	10	Cuadrícula y catálogo de terrenos	Implementado
`Mapa.dentro(f, c)`	16	Comprueba límites de la cuadrícula	Implementado
`Mapa.es_transitable(f, c)`	25	`dentro` y `tipos[terreno]["transitable"]`	Implementado
`Mapa.costo(f, c)`	34	Costo de entrar a la celda; incluye `assert` de transitabilidad	Implementado
`Mapa.costo_minimo()`	44	Mínimo costo entre terrenos transitables	Implementado
`Mapa.vecinos(f, c)`	53	Lista de `(nombre_dir, f2, c2)` transitables en el orden de `DIRECCIONES`	Implementado

8.9 `escenario/modelos.py`

Elemento	Línea	Responsabilidad	Estado
`Escenario`	10	Datos validados del escenario	Implementado
`Escenario.estado_inicial()`	20	Construye `Estado`; si hay portador, `recurso_pos = None`; ordena unidades por `id`	Implementado

8.10 `escenario/cargador.py`

Elemento	Línea	Responsabilidad	Estado
`PROFUNDIDAD_POR_DEFECTO`	9	Valor `3`	Informativo
`ErrorEscenario`	12	Excepción con lista `errores`	Implementado
`ErrorEscenario.__init__(errores)`	14	Guarda la lista y compone el mensaje	Implementado
`construir_escenario(datos)`	19	Valida y convierte el dict en `Escenario`	Implementado
`cargar_escenario(ruta)`	49	Lee JSON y delega en `construir_escenario`	Implementado

8.11 `escenario/validador.py`

Elemento	Línea	Responsabilidad	Estado
`OBLIGATORIOS`	3	Claves obligatorias de la raíz	Informativo
`_es_entero(x)`	7	Entero y no booleano	Implementado
`_es_numero(x)`	11	Número y no booleano	Implementado
`validar_datos(d)`	15	Devuelve lista de errores; lista vacía si es válido	Implementado

Comportamiento verificado: si la raíz no es `dict` o faltan claves obligatorias, retorna de inmediato. En caso contrario acumula todos los errores detectados. La función auxiliar interna `transitable(p)` devuelve `True` ante `KeyError` / `IndexError` / `TypeError` para no duplicar errores ya reportados.

8.12 `juego/estado.py`

Elemento	Línea	Responsabilidad	Estado
`Unidad`	7	Entidad inmutable de unidad	Implementado
`Estado`	15	Estado inmutable del juego	Implementado
`Estado.unidad(uid)`	21	Busca una unidad por id; `None` si no existe	Implementado
`Estado.unidades_de(bando)`	24	Lista de unidades del bando	Implementado
`Estado.posicion_recurso()`	27	Posición del recurso en suelo o con el portador	Implementado
`Accion`	36	Acción inmutable	Implementado

`Estado.posicion_recurso()` no se invoca desde los módulos revisados (solo se define). No se encontró uso en `app.py`, `reglas.py`, `tablero.py` ni en las pruebas.

8.13 `juego/reglas.py`

Reglas declaradas en el docstring del módulo: R1–R7 (una unidad por turno; no compartir celda salvo interceptación; recogida automática; interceptación del portador enemigo; victoria por base propia o por dejar al rival sin unidades; costo de movimiento = costo del terreno destino, esperar cuesta 0; siempre existe ESPERAR).

Elemento	Línea	Responsabilidad	Estado
`Reglas.__init__(escenario)`	19	Guarda `mapa` y `bases`	Implementado
`Reglas.acciones_legales(estado)`	23	Acciones del bando en turno, en orden determinista	Implementado
`Reglas.costo(estado, accion)`	43	`0` si ESPERAR; costo del destino si MOVER	Implementado
`Reglas.aplicar(estado, accion)`	49	Función de transición sin mutar el estado anterior	Implementado
`Reglas.ganador(estado)`	75	Devuelve `"A"`, `"B"` o `None`	Implementado
`Reglas.es_terminal(estado)`	89	`ganador(estado) is not None`	Implementado

Detalles verificables de `acciones_legales`:

- Construye `ocupadas` como `{(fila, columna): unidad}`.
- Recorre las unidades del bando en turno.
- Para cada dirección: descarta destinos no transitables; descarta destinos ocupados, salvo que el ocupante sea el portador enemigo (excepción de interceptación).
- Agrega `Accion(u.id, "ESPERAR")` por cada unidad.

Detalles verificables de `aplicar`:

- Copia la lista de unidades, `recurso_pos` y `portador`.
- Si es MOVER y hay una víctima en el destino, la elimina y deja el recurso en el destino con `portador = None`.
- Reemplaza la unidad por una nueva instancia (las entidades son inmutables).
- Aplica recogida automática si el recurso está en el destino.
- Alterna el turno y devuelve un `Estado` con unidades ordenadas por `id`.

Condiciones previas declaradas: `accion in acciones_legales(estado)`. Si se violan, el uso de `next(...)` sin valor por defecto puede producir `StopIteration` al buscar la unidad.

8.14 `juego/problema_navegacion.py`

Elemento	Línea	Responsabilidad	Estado
`ProblemaNavegacion`	4	Problema de un agente; estado `(fila, columna)`	Implementado
`ProblemaNavegacion.__init__(mapa, inicio, objetivo)`	8	Convierte `inicio` y `objetivo` a tuplas	Implementado

Elemento	Línea	Responsabilidad	Estado
`ProblemaNavegacion.estado_inicial()`	11	Devuelve `inicio`	Implementado
`ProblemaNavegacion.es_objetivo(s)`	14	Igualdad con `objetivo`	Implementado
`ProblemaNavegacion.sucesores(s)`	17	`(nombre_accion, estado, costo)` desde `Mapa.vecinos`	Implementado

Ejemplo real de `sucesores` (código íntegro):

```
return [(n, (f, c), self.mapa.costo(f, c)) for n, f, c in self.mapa.vecinos(*s)]
```

8.15 `verificacion/verificador.py`

Elemento	Línea	Responsabilidad	Estado
`verificar_camino(mapa, inicio, objetivo, resultado)`	4	Devuelve lista de `(descripción, ok, detalle)`	Implementado

Comprobaciones realizadas (seis):

1. El camino comienza en la celda inicial.
2. Todas las celdas están dentro del mapa.
3. Ninguna celda es obstáculo (todas transitables).
4. Cada movimiento es un paso ortogonal de distancia Manhattan 1.
5. El costo reportado coincide con la suma de costos de las transiciones (tolerancia `1e-9`).
6. La última celda es el objetivo.

8.16 `interfaz/app.py`

Constantes del módulo:

Constante	Línea	Valor	U
`TAREAS`	21	`{"Búsqueda de ruta": "busqueda", "Partida por turnos": "partida", "Decisión adversarial": "adversarial"}`	Mapeo nombre → clav interna
`MODO_A_TAREA`	22	Inverso de `TAREAS`	Mapeo → nom visible
`METRICAS`	23	Seis métricas de búsqueda	Opcion los despleg de métr tareas búsque
`METRICAS_ADV`	26	Nodos evaluados/generados/podados, valor y profundidad	Despleg de métr la tarea adversa
`HEURISTICAS`	32	`("manhattan", "propia")`	Heuríst ofrecid interfaz (euclidi existe e `a_estr` pero no expone

Constante	Línea	Valor	U
`MAX_JUGADAS_AUTO`	33	`300`	Tope de jugadas auto-ju
`EJEMPLO`	34	Ruta a `escenarios/ejemplo.json`	Escena ejemplo

Funciones de módulo:

Elemento	Línea	Responsabilidad
`fmt(v)`	37	Formatea `None` como `–`, booleanos como `Sí` / `No`, flotantes redondeados a 3 decimales
`_desc_accion(a)`	47	Descripción legible de una acción (`MOVER (fila,col)`, `RECOGER`, `ESPERAR`, `INTERCEPTAR`)

Clase `App(tk.Tk)` (declarada en la línea 54):

Elemento	Línea	Responsabilidad	Estado
`__init__`	55	Inicializa atributos, tema, construcción de UI, frame de tarea y estado	Implementado
`_tema`	73	Configura estilos `ttk` (tema oscuro)	Implementado
`_texto`	159	Crea caja de texto con etiquetas de color	Implementado
`_construir`	171	Barra superior, panel dividido, tablero y pestañas	Implementado
`_tab_control`	229	Pestaña Control con misión y formularios	Implementado
`_f_busqueda`	254	Formulario de búsqueda (algoritmo, unidad, objetivo, heurística, `k`, opciones)	Implementado
`_f_partida`	305	Formulario de partida: control por bando (`Humano`, `Alfa-Beta`, `Minimax`, `Azar (provisional)`) y botones	Implementado
`_f_adversarial`	335	Formulario adversarial: algoritmo, profundidad 1–8, orden de acciones y botones «Evaluar estado»/«Evaluar y jugar»	Implementado
`_tab_resultados`	361	Tabla de métricas y texto de resultados	Implementado
`_tab_estadisticas`	373	Resumen, historial, gráfico, comparación de heurísticas	Implementado
`_tab_comparacion`	420	Comparación de algoritmos (o de `MINIMAX` vs `ALFA_BETA`), selector de heurística de A*	Implementado
`_tab_escenario`	449	Resumen del escenario y log	Implementado
`_poner_estado`	459	Escribe la barra de estado	Implementado
`log`	462	Agrega línea al log	Implementado
`_tarea`	466	Clave interna de la tarea seleccionada	Implementado
`_modo_json`	469	`True` si el modo es JSON	Implementado
`_es_astar`	472	`True` si el algoritmo seleccionado normaliza a `A_ESTRELLA`	Implementado
`_es_adversarial`	475	`True` si la tarea activa es `adversarial`	Implementado
`_metricas_activas`	481	Conjunto de métricas habilitadas (`METRICAS` o `METRICAS_ADV` según tarea)	Implementado
`_sincronizar_metricas`	484	Reajusta los desplegables de métricas al cambiar de tarea	Implementado
`_actualizar_visibles`	495	Muestra heurística/comparación solo con A*, `k` solo con Beam Search y las métricas adversariales en su tarea	Implementado
`cargar_dialogo`	512	Diálogo de selección de JSON	Implementado
`cargar_ruta`	518	Carga validando primero; conserva el escenario anterior ante error	Implementado

Elemento	Línea	Responsabilidad	Estado
`_mostrar_errores`	531	Muestra errores de validación	Implementado
`_aplicar_escenario`	539	Crea `Reglas`, estado, ajusta controles, limpia y redibuja	Implementado
`_resumen_escenario`	563	Texto con detalles del escenario	Implementado
`_dibujar_leyenda`	577	Leyenda de terrenos y símbolos	Implementado
`cambiar_modo`	606	Aplica modo JSON/exploración, bloqueo y ejecución automática	Implementado
`_aplicar_prueba`	622	Lee `prueba` y ajusta controles	Implementado
`_bloquear`	651	Habilita/deshabilita controles	Implementado
`_elegir_tarea`	659	Cambia la tarea activa	Implementado
`_cambio_tarea`	666	Redibuja tras cambio de tarea	Implementado
`_mostrar_frame_tarea`	672	Muestra el formulario de la tarea activa	Implementado
`_cambio_parametros`	679	Limpia capas y redibuja ante cambios de parámetros	Implementado
`_inicio_busqueda`	685	Posición de la unidad de inicio	Implementado
`_objetivo_busqueda`	691	Objetivo si está dentro del mapa	Implementado
`refrescar_tablero`	698	Compone capas y actualiza tablero, turno y costos	Implementado
`_limpiar_vista`	728	Limpia capas visuales	Implementado
`_hover_celda`	732	Información de la celda bajo el cursor	Implementado
`clic_tablero`	757	Despacha clics según tarea	Implementado
`_es_humano`	777	Indica si el bando es humano	Implementado
`_clic_partida`	780	Selección de unidad y movimiento en partida	Implementado
`_reiniciar_partida`	801	Restaura el estado inicial y contadores	Implementado
`_aplicar_accion`	810	Aplica una acción, acumula costo, registra y comprueba victoria	Implementado
`esperar`	829	Acción ESPERAR para la unidad seleccionada	Implementado
`turno_ia`	838	Elige la acción del bando en turno: `Alfa-Beta` (orden `heuristico`), `Minimax` (orden por defecto) o aleatoria (`Azar`); usa `var_prof` y ante `None` recurre a `rng.choice`	Implementado
`auto_jugar`	865	Alterna el auto-juego	Implementado
`_auto_tick`	873	Avanza el auto-juego cada 250 ms, con tope de 300 jugadas; se detiene en estado terminal o ante turno humano	Implementado
`ejecutar`	892	Despacha según tarea	Implementado
`_problema`	913	Construye y valida `ProblemaNavegacion`	Implementado
`_parametros`	932	Devuelve `{"k": int, "heuristica": str}`	Implementado
`_correr`	939	Ejecuta, verifica, agrega al historial y devuelve `(res, checks)`	Implementado
`_ejecutar_busqueda`	951	Ejecuta búsqueda, muestra resultados; si es A*, compara heurísticas	Implementado
`comparar_todos`	971	Ejecuta todos los algoritmos de búsqueda registrados; en tarea adversarial deriva a `comparar_adversariales`	Implementado
`comparar_adversariales`	991	Corre `MINIMAX` y `ALFA_BETA` con el mismo orden sobre el estado actual y cambia a la pestaña Comparación	Implementado
`_mostrar_comparacion_adversarial`	1009	Rellena la tabla adversarial, resume acción/valor/evaluaciones y selecciona la primera fila	Implementado

Elemento	Línea	Responsabilidad	Estado
`_encabezados_cmp`	1027	Alterna los encabezados de la tabla de comparación (búsqueda / adversarial)	Implementado
`verificar`	1032	Re-verifica la última ejecución	Implementado
`_correr_adversarial`	1046	Invoca `minimax` / `alfa_beta` con profundidad y orden de la UI, registra la corrida en el historial y devuelve `ResultadoAdversarial`	Implementado
`_resaltar_accion`	1074	Marca la unidad que mueve y la celda destino de la acción recomendada	Implementado
`_ejecutar_adversarial`	1083	«Evaluar estado»: corre el algoritmo, muestra métricas y resalta la acción sin aplicarla (rechaza estados terminales)	Implementado
`_evaluar_y_aplicar_adversarial`	1104	«Evaluar y jugar»: corre el algoritmo y aplica la acción recomendada vía `_aplicar_accion`	Implementado
`_mostrar_resultado_adversarial`	1117	Rellena la tabla de resultados (10 filas), el texto libre y el JSON de `ResultadoAdversarial`	Implementado
`_limpiar_resultados`	1146	Limpia resultados, comparación e historial de heurísticas	Implementado
`_mostrar_resultado`	1158	Rellena métricas, camino, verificación y JSON	Implementado
`_mostrar_comparacion`	1182	Rellena tabla de comparación y resumen	Implementado
`_seleccion_comparacion`	1206	Muestra el camino/acción del elemento seleccionado en el tablero	Implementado
`_graficar_comparacion`	1219	Gráfico de barras por algoritmo	Implementado
`_refrescar_estadisticas`	1227	Historial, resumen y gráfico	Implementado
`_agrupar`	1249	Agrupa historial por algoritmo	Implementado
`_graficar_estadisticas`	1255	Barras por heurística si A* está activo; si no, promedio por algoritmo	Implementado
`_comparar_heuristicas`	1272	Ejecuta A* con cada heurística y guarda resultados	Implementado
`_refrescar_comparacion_heur`	1283	Rellena tabla de heurísticas y redibuja	Implementado
`exportar_csv`	1297	Exporta el historial a CSV	Implementado
`borrar_historial`	1315	Vacía historial y comparación de heurísticas	Implementado

Comportamientos verificables de la interfaz:

- La heurística (`cb_heur`) y la tabla/checkboxes de comparación de heurísticas se muestran solo cuando el algoritmo seleccionado normaliza a `A_ESTRELLA` .
- El control `k` se muestra solo cuando el algoritmo seleccionado es `BEAM_SEARCH` .
- Al ejecutar A* se comparan ambas heurísticas: se reutiliza el resultado de la heurística seleccionada y se ejecuta A* con la otra.
- La pestaña Comparación conserva la ejecución de todos los algoritmos y permite elegir la heurística usada por A* (`var_heur_cmp`).
- Cada llamada a `_correr` agrega una entrada al historial; por tanto, una ejecución de A* con comparación de heurísticas agrega una entrada por cada heurística ejecutada (la seleccionada se reutiliza, la otra se ejecuta y registra).
- En la tarea **Decisión adversarial**, el botón «Comparar» llama a `comparar_adversariales()` , que solo corre `MINIMAX` y `ALFA_BETA` con el mismo `orden_acciones` y reetiqueta las columnas de la tabla vía `_encabezados_cmp(adv=True)` .
- Las corridas adversariales se registran en el mismo historial que las de búsqueda, reutilizando `estados_generados/estados_expandidos` para `nodos_generados/nodos_evaluados` y añadiendo los campos `valor` , `nodos_podados` , `profundidad` y `orden` dentro de `parametros` (JSON).
- Ambos botones de evaluación rechazan el estado si `Reglas.es_terminal` devuelve `True` (mensaje «Fin de partida»).
- Si `minimax` / `alfa_beta` devuelven `accion = None` , `turno_ia` cae en una acción aleatoria legal; «Evaluar estado» solo informa «no se propuso ninguna acción».

8.17 `interfaz/tablero.py`

Elemento	Línea	Responsabilidad	Estado
Constantes de paleta	5–17	Colores del tema oscuro	Informativo

Elemento	Línea	Responsabilidad	Estado
`_rgb` , `_hex` , `mezclar` , `aclarar` , `oscurecer`	20–40	Utilidades de color	Implementado
`contraste(hex_color)`	43	Color de texto legible sobre un fondo	Implementado
`colores_terreno(mapa)`	52	Color por tipo; derivado del costo para tipos no fijos	Implementado
`fmt_num(v)`	70	Formato numérico compacto	Implementado
`Tablero`	74	`tk.Canvas` que dibuja el estado	Implementado
`Tablero.__init__`	75	Enlaza eventos y callbacks	Implementado
`Tablero.mostrar`	85	Guarda escenario, estado y capas; redibuja	Implementado
`Tablero.limpiar`	90	Borra la vista	Implementado
`_rect` , `_centro` , `_esquina` , `_corchetes` , `_placa`	95–119	Geometría y elementos visuales	Implementado
`redibujar`	130	Dibuja terreno, bases, capas, recurso y unidades	Implementado
`_dibujar_hover` , `_poner_hover`	301, 309	Resaltado bajo el cursor	Implementado
`_celda_en` , `_movimiento` , `_clic`	317–330	Conversión de píxeles a celda y despacho de eventos	Implementado

Capas aceptadas por `mostrar` : `camino` , `explorados` , `inicio` , `objetivo` , `seleccion` , `destinos` , `turno_marcador` .

8.18 `interfaz/graficas.py`

Elemento	Línea	Responsabilidad	Estado
`_fmt` , `_rect`	8, 12	Utilidades de formato y rectángulos	Implementado
`barras(canvas, datos, titulo="")`	18	Dibuja barras y rejilla; ignora valores `None` ; si el canvas es menor a 40×60 no dibuja	Implementado

8.19 `experimentos/`

Archivo / elemento	Responsabilidad	Estado
`generar_casos_minimos.py`	Genera `caso01` , `caso05` , `caso06a` , `caso06b` en `escenarios/casos_minimos`	Implementado
`generar_casos_minimos.matriz` , `poner` , `rect` , `unidad` , `escenario` , `guardar`	Utilidades de construcción	Implementado
`generar_casos_minimos.caso1` , `caso5` , `caso6`	Definición de los casos	Implementado
`generar_casos_robustez.py`	Genera `caso02` , los escenarios `robustez/r01-r10` y los `invalidos/inv01-inv12` con semillas fijas	Implementado
`caso06_estado_compuesto.py`	Imprime comparación de estados de los casos 6a y 6b	Implementado
`ejecutar_experimentos_busqueda.py`	Ejecuta los casos mínimos 1–5, los robustos `R01-R08` y los adversariales 7–10; escribe `documentacion/resultados_experimentos_busqueda.md`	Implementado
`ejecutar_experimentos_busqueda.ejecutar_caso`	Corre un caso de navegación, rechaza objetivos no transitables (`Verif. = –`)	Implementado
`ejecutar_experimentos_busqueda.ejecutar_caso_adversarial`	5 iteraciones por configuración; promedia tiempo y reporta nodos	Implementado
`ejecutar_experimentos_busqueda.CASOS_ROBUSTOS` , `CASOS_ADVERSARIALES`	Cataloga los 8 escenarios robustos y los 4 adversariales (profundidad y `orden_acciones`)	Implementado

Archivo / elemento	Responsabilidad	Estado
`ejecutar_casos_adversariales.py`	Reporte por consola de los casos 7, 8, 9 y 10 (comparación MM/AB, órdenes y profundidades)	Implementado
`ejecutar_casos.py`	Tabla «esperado vs. obtenido» a partir de `esperados.json`	**No ejecutable**: importa `algoritmos.adversarial` y `verificacion.verificador_advers` que no existen
`esperados.json`	Especificación previa de C01–C10, R01–R10 y V01–V12	Implementado (su clave `orden` n coincide con `orden_acciones` de `minimax.py`)

Observaciones verificables:

- `generar_casos_minimos.py` escribe en `escenarios/casos_minimos` (guion bajo), mientras que la carpeta versionada es `escenarios/casos minimos` (espacio); `generar_casos_robustez.py` escribe en `robustez` / `invalidos` frente a las carpetas versionadas `casos_robustos` / `casos_invalidos` . Ejecutar esos generadores crearía carpetas distintas de las usadas por las pruebas y los escenarios existentes.
- `ejecutar_experimentos_busqueda.py` sí apunta a `escenarios/casos minimos` y por eso es el generador oficial de resultados.

9. Parámetros, tipos, retornos, excepciones y efectos secundarios

Solo se documentan datos verificables en el código. Las anotaciones de tipo son las declaradas; cuando el código no anota, se indica «por uso».

9.1 Funciones del cargador

Función	Parámetros	Retorno	Excepciones	Efectos secundarios
`validar_datos(d)`	`d` : cualquier objeto	`list[str]` (vacía si válido)	No declaradas; maneja internamente tipos inesperados	Ninguno; no modifica `d`
`construir_escenario(datos)`	`datos` : `dict`	`Escenario`	`ErrorEscenario` si hay errores de validación	Ninguno
`cargar_escenario(ruta)`	`ruta` : ruta de archivo	`Escenario`	`ErrorEscenario` ante JSON mal formado o error de E/S	Lectura de archivo

9.2 Funciones de reglas

Función	Parámetros	Retorno	Excepciones	Efectos secundarios
`Reglas.__init__(escenario)`	`escenario` : `Escenario`	instancia	No	Guarda referencias
`Reglas.acciones_legales(estado)`	`estado` : `Estado`	`list[Accion]`	No	Ninguno
`Reglas.costo(estado, accion)`	`estado` , `accion`	número (`0` o costo del destino)	Precondición: acción legal	Ninguno
`Reglas.aplicar(estado, accion)`	`estado` , `accion`	`Estado` nuevo	Potencial `StopIteration` si la acción no corresponde a una unidad existente (precondición)	Ninguno; no muta el estado anterior
`Reglas.ganador(estado)`	`estado`	`"A"` , `"B"` o `None`	Precondición: estado consistente; un `portador` inexistente produciría `AttributeError`	Ninguno

Función	Parámetros	Retorno	Excepciones	Efectos secundarios
`Reglas.es_terminal(estado)`	`estado`	`bool`	Las de `ganador`	Ninguno

9.3 Funciones de búsqueda

Función	Parámetros	Retorno
`ejecutar_algoritmo(nombre, problema, **params)`	`nombre` : clave del registro; `problema` : objeto con `estado_inicial` , `es_objetivo` , `sucesores` ; `params` : arbitrarios	`ResultadoBusqued`
`bfs(problema, **params)`	`problema`	`ResultadoBusqued`
`dfs(problema, **params)`	`problema`	`ResultadoBusqued`
`ucs(problema, **params)`	`problema`	`ResultadoBusqued`
`a_estrella(problema, **params)`	`problema` ; `params["heuristica"]` opcional (`"manhattan"` por defecto)	`ResultadoBusqued`
`beam_search(problema, **params)`	`problema` ; `params["k"]` (por defecto `2` , forzado a `≥ 1`); `params["heuristica"]` (por defecto `"manhattan"`)	`ResultadoBusqued` (`camino=[]` si no ruta)
`minimax(estado, reglas, escenario, profundidad_maxima=3, orden_acciones="estandar")`	`estado` : `Estado` ; `reglas` : `Reglas` ; `escenario` : `Escenario` ; profundidad entera `≥ 1` ; orden en `("estandar","inverso","heuristico")`	`ResultadoAdversa`
`alfa_beta(estado, reglas, escenario, profundidad_maxima=3, orden_acciones="estandar")`	Ídem `minimax`	`ResultadoAdversa` (`algoritmo="ALFA` `nodos_podados` >
`ResultadoAdversarial.a_dict()`	—	`dict` serializable
`h_manhattan(s, problema)`	`s` : celda; `problema`	número
`h_propia(s, problema)`	`s` : celda; `problema`	número
`h_euclidiana(s, problema)`	`s` : celda; `problema`	número

Función	Parámetros	Retorno
`obtener_heuristica(nombre)`	`nombre` : cualquier valor convertible a cadena	función de heurís
`reconstruir_camino(padre, meta)`	`padre` : dict; `meta` : estado	`(list, número)`

9.4 Funciones de mapa y modelo

Función	Parámetros	Retorno	Excepciones	Efectos secundarios
`Mapa.dentro(f, c)`	enteros	`bool`	No	Ninguno
`Mapa.es_transitable(f, c)`	enteros	`bool`	`KeyError` si el nombre de terreno no está en `tipos` (escenario inconsistente)	Ninguno
`Mapa.costo(f, c)`	enteros	número	`AssertionError` si la celda no es transitable	Ninguno
`Mapa.costo_minimo()`	—	número	`ValueError` si no hay terrenos transitables (`min` sobre vacío)	Ninguno
`Mapa.vecinos(f, c)`	enteros	`list[(str, int, int)]`	Las de `es_transitable`	Ninguno
`Escenario.estado_inicial()`	—	`Estado`	No	Ninguno
`Estado.unidad(uid)`	`str`	`Unidad` o `None`	No	Ninguno
`Estado.unidades_de(bando)`	`str`	`list[Unidad]`	No	Ninguno
`Estado.posicion_recurso()`	—	tupla o `None`	`AttributeError` si `portador` no corresponde a una unidad	Ninguno

9.5 Verificación

Función	Parámetros	Retorno	Excepciones	Efectos secundarios
`verificar_camino(mapa, inicio, objetivo, resultado)`	`mapa` ; `inicio` , `objetivo` : tuplas; `resultado` : `ResultadoBusqueda` con `exito=True`	`list[(str, bool, str)]`	Las de `mapa.dentro` , `mapa.es_transitable` , `mapa.costo` si el camino es inválido	Ninguno

9.6 Interfaz

Elemento	Entradas verificables	Salidas / efectos
`App()`	—	Abre ventana; crea widgets y atributos de estado
`App.cargar_ruta(ruta)`	ruta de archivo	Carga escenario o muestra errores; actualiza controles, leyenda y log; puede ejecutar automáticamente
`App.ejecutar()`	estado de la UI	Ejecuta búsqueda, partida o adversarial; puede mostrar diálogos
`App.comparar_todos()`	escenario y objetivo actuales	Ejecuta todos los algoritmos de búsqueda; en tarea adversarial ejecuta `comparar_adversariales` ; agrega entradas al historial y muestra comparación

Elemento	Entradas verificables	Salidas / efectos
`App.comparar_adversariales()`	estado actual y `orden_acciones`	Ejecuta `MINIMAX` y `ALFA_BETA`, agrega dos entradas al historial y abre la pestaña Comparación
`App._ejecutar_adversarial()`	estado actual, algoritmo, profundidad y orden de la UI	Evalúa sin aplicar; rechaza estados terminales; resalta la acción recomendada
`App._evaluar_y_aplicar_adversarial()`	ídem	Evalúa y aplica la acción vía `_aplicar_accion`; puede avanzar el turno
`App.turno_ia()`	control del bando y estado	Elige y aplica una acción (Alfa-Beta, Minimax o aleatoria)
`App.exportar_csv()`	historial en memoria	Abre diálogo de guardado; escribe el CSV en la ruta elegida con las claves del primer registro del historial
`App.borrar_historial()`	—	Vacía historial, comparación de heurísticas y refresca vistas

Diálogos observables: `messagebox.showinfo`, `showerror`, `showwarning` y `filedialog.asksaveasfilename` / `askopenfilename`.

10. Configuración, instalación y ejecución

10.1 Requisitos

Requisito	Evidencia
Python 3.9 o superior	`README.md`, línea 9
`tkinter` incluido en la instalación de Python	`README.md`, línea 9
En Ubuntu/Debian: `sudo apt install python3-tk`	`README.md`, línea 9
Sin dependencias de terceros	No hay archivo de dependencias; todas las importaciones son de la biblioteca estándar

10.2 Variables de entorno

No se detectó lectura de variables de entorno (`os.environ`, `os.getenv`, `dotenv`) ni uso de argumentos de línea de comandos (`argparse`, `sys.argv`) en el código del proyecto. La configuración se realiza mediante:

- el archivo JSON de escenario cargado desde la interfaz;
- los controles de la interfaz (algoritmo, heurística, `k`, unidad, objetivo, tarea, modo).

10.3 Instalación

No se requiere instalación de paquetes. Basta con disponer de Python y `tkinter`. No hay instalador, empaquetado ni script de configuración.

10.4 Ejecución (respaldada por archivos existentes)

Desde la raíz del repositorio:

```
python3 main.py
```

En el entorno Windows de elaboración se verificó que el intérprete disponible responde a `python` (3.12.0 en la versión inicial del manual; **3.14.0 en la re-ejecución del 2026-10-20**). La forma equivalente es:

```
python main.py
```

> No se ejecutó la interfaz gráfica durante la elaboración de este manual; la instrucción se deriva de `main.py` y del `README.md`.

10.5 Ejecución de pruebas

```
python3 tests/test_paso1_2.py
```

Forma equivalente verificada durante la elaboración:

```
python -m unittest discover -s tests -v
```

10.6 Generación de escenarios de ejemplo

```
python3 experimentos/generar_casos_minimos.py
```

Advertencia verificable: el script escribe en `escenarios/casos_minimos`, mientras que los escenarios versionados están en `escenarios/casos minimos`.

10.7 Ejecución del experimento del caso 6

```
python3 experimentos/caso06_estado_compuesto.py
```

Este script imprime por consola posiciones, igualdad/hash de estados, portador, recurso y acciones legales de los casos 6a y 6b.

10.8 Experimentos (reporte de resultados)

```
python experimentos/ejecutar_experimentos_busqueda.py
python experimentos/ejecutar_casos_adversariales.py
python experimentos/generar_casos_robustez.py
```

- `ejecutar_experimentos_busqueda.py` (el oficial): corre los casos mínimos 1–5, los robustos `R01–R08` y los adversariales 7–10, y reescribe `documentacion/resultados_experimentos_busqueda.md` con las tres secciones. Los adversariales se repiten 5 veces y reportan tiempo promedio.
- `ejecutar_casos_adversariales.py`: reporte detallado por consola de los casos 7–10 (acciones, valor, nodos generados/evaluados/podados y tiempo por configuración).
- `generar_casos_robustez.py`: regenera escenarios con semillas fijas; escribe en carpetas con nombres distintos a las versionadas (véase §8.19).

```
python experimentos/ejecutar_casos.py
```

No ejecutable: falla con `ModuleNotFoundError: No module named 'algoritmos.adversarial'` (y `verificacion.verificador_adversarial`), módulos que no existen en el repositorio. Véase 15.2.

10.9 Modos de la interfaz

Modo	Comportamiento verificado
Modo JSON (por defecto, `var_modo = "json"`)	Aplica `prueba` y bloquea controles; si hay `prueba`, ejecuta automáticamente la tarea
Modo exploración	Permite elegir libremente tarea, algoritmo, unidad y objetivo

11. Pruebas existentes y mecanismos de validación

11.1 Pruebas automatizadas

Se ejecutó la suite completa durante la elaboración con:

```
python -m unittest discover -s tests -v
```

Resultado obtenido: 25 pruebas ejecutadas, 25 correctas (`OK`), en 0.153 s, con Python 3.14.0 (re-ejecución del 2026-10-20; la versión inicial del manual reportaba 17 pruebas en 0.029 s con Python 3.12.0).

Archivo	Clase	Prueba	Qué comprueba (según el código)	C
`tests/test_paso1_2.py`	`TestCarga`	`test_valido`	Dimensiones, costo `(2,2)=7` , costo mínimo `1` , profundidad `4`	C
`tests/test_paso1_2.py`	`TestCarga`	`test_errores_multiples`	Al menos 5 errores ante varias inconsistencias	C
`tests/test_paso1_2.py`	`TestCarga`	`test_unidad_sobre_muro`	Error «no transitable»	C
`tests/test_paso1_2.py`	`TestCarga`	`test_costo_no_positivo`	Error «costo positivo»	C
`tests/test_paso1_2.py`	`TestCarga`	`test_dimensiones_inconsistentes`	Error por filas incoherentes	C
`tests/test_paso1_2.py`	`TestJuego`	`test_hashable_e_inmutable`	`Estado` igual y hashable	C
`tests/test_paso1_2.py`	`TestJuego`	`test_acciones_y_sucesor`	Acciones del bando en turno y alternancia de turno	C
`tests/test_paso1_2.py`	`TestJuego`	`test_estado_compuesto`	Misma posición de unidad, estados distintos según portador	C
`tests/test_paso1_2.py`	`TestJuego`	`test_unidad_que_empieza_sobre_el_recurso_lo_recoge`	Al inicio, la unidad sobre el recurso queda como portadora	C
`tests/test_paso1_2.py`	`TestJuego`	`test_recoger_recurso`	Recogida automática al pisar el recurso	C
`tests/test_paso1_2.py`	`TestJuego`	`test_victoria`	Victoria al portador en base propia	C
`tests/test_paso1_2.py`	`TestJuego`	`test_interceptar`	Interceptación del portador enemigo	C
`tests/test_paso1_2.py`	`TestJuego`	`test_no_atraviesa_muro_ni_ocupadas`	Destinos legales transitables y no ocupados	C
`tests/test_paso3_ucs.py`	`TestUCS`	`test_ucs_ejecucion_y_verificacion`	Éxito, métricas positivas y verificación del camino	C
`tests/test_paso3_ucs.py`	`TestUCS`	`test_caso2_bfs_vs_ucs_diferencia`	Caminos distintos; BFS menos movimientos; UCS menor costo	C
`tests/test_paso4_a_estrella.py`	`TestAEstrella`	`test_admisibilidad_heuristicas`	Heurísticas `0` en meta y `h_propia >= h_manhattan` en inicio	C
`tests/test_paso4_a_estrella.py`	`TestAEstrella`	`test_caso3_obstaculo_y_heuristica`	Camino verificado y rodeo del muro	C
`tests/test_paso4_a_estrella.py`	`TestAEstrella`	`test_caso4_comparacion_heuristicas`	Mismo costo óptimo; `propia` expande menos que `manhattan`	C

Archivo	Clase	Prueba	Qué comprueba (según el código)	
tests/test_beam_search.py	TestBeamSearch	test_k_1_2_4_8_devuelven_camino_valido_en_caso5	k=1,2 → sin solución (exito=False , camino=[]); k=4,8 → ruta verificada; siempre maximo_frontera ≤ k	
tests/test_beam_search.py	TestBeamSearch	test_sin_solucion_termina_sin_camino	Barrera de muros 3×3: termina sin excepción y sin camino	
tests/test_minimax.py	TestBusquedaAdversarial	test_caso7_intercepta_portador_adyacente	Alfa-Beta (prof 3) elige A1 → (8,9) ; el sucesor elimina a B1 y A1 pasa a ser portadora	
tests/test_minimax.py	TestBusquedaAdversarial	test_caso8_minimax_y_alfa_beta_equivalentes	Misma acción y mismo valor; alfa-beta evalúa menos nodos que minimax	
tests/test_minimax.py	TestBusquedaAdversarial	test_caso9_orden_cambia_trabajo_no_valor	Los tres órdenes dan el mismo valor (dif. < 1e-9), acción no nula y nodos evaluados > 0	
tests/test_minimax.py	TestBusquedaAdversarial	test_caso10_profundidades_1_a_4_limitan_busqueda	Profundidades 1–4 respetadas, acción y métricas no vacías	
tests/test_minimax.py	TestBusquedaAdversarial	test_rechaza_profundidad_invalida	Profundidades 0 , -1 y 1.5 lanzan ValueError	

Nota: la prueba test_admisibilidad_heuristicas no comprueba numéricamente la admisibilidad global de las heurísticas; comprueba los valores en la meta y una relación de orden en el inicio, según el código de la prueba.

Nota de alineación: test_k_1_2_4_8_devuelven_camino_valido_en_caso5 fue corregida el 2026-10-20 para reflejar el comportamiento **medido** de caso05_beam_search.json (haz estrecho atrapado en la copa: k=1,2 sin solución; k=4,8 con ruta verificable). En la versión anterior del manual y de la prueba se esperaba que k=2 encontrara ruta.

11.2 Mecanismos de validación en producción

Mecanismo	Ubicación	Función
Validación de escenarios	escenario/validador.py	Acumula errores de estructura, dimensiones, terrenos, bases, recurso, unidades, turno y portador
Carga «todo o nada»	escenario/cargador.py , App.cargar_ruta	No reemplaza el escenario anterior si el nuevo es inválido
Aserción de costo	Mapa.costo	assert de transitabilidad antes de leer el costo
Verificación de caminos	verificacion/verificador.py	Seis comprobaciones independientes del algoritmo
Verificación en interfaz	App._correr , App.verificar	Ejecuta verificar_camino y muestra el resultado
Comprobación de algoritmo implementado	App._ejecutar_busqueda	Rechaza algoritmos no registrados
Rechazo de estado terminal adversarial	App._ejecutar_adversarial , _evaluar_y_aplicar_adversarial , comparar_adversariales , turno_ia	Reglas.es_terminal → diálogo «Fin de partida» / retorno temprano

Mecanismo	Ubicación	Función
Validación de parámetros adversariales	`algoritmos/minimax._buscar`	`ValueError` si `profundidad_maxima < 1` /no entero o `orden_acciones` fuera de `estandar/inverso/heuristico`
Fallback de acción adversarial	`App.turno_ia`	Si la búsqueda devuelve `accion = None` , se elige una acción aleatoria legal

11.3 Pruebas no existentes

- No hay pruebas automatizadas de la interfaz gráfica.
- No hay pruebas de `interfaz/tablero.py` , `interfaz/graficas.py` ni de los flujos de `App` (aunque parte de la lógica se ejerce indirectamente desde los tests de algoritmos y reglas).
- En la versión inicial del manual no había pruebas de Beam Search ni de Minimax/alfa-beta; **hoy existen** (`tests/test_beam_search.py` , 2 pruebas; `tests/test_minimax.py` , 5 pruebas). No hay pruebas que comparen Beam Search contra los costos óptimos de A*/UCS (lo que haría cuantificable su suboptimalidad).
- No hay pruebas automatizadas que ejerzan la interfaz adversarial (`_ejecutar_adversarial` , `_evaluar_y_aplicar_adversarial` , `comparar_adversariales`).
- `experimentos/esperados.json` no está conectado a ninguna prueba automatizada (`ejecutar_casos.py` no es ejecutable).
- No hay medición de cobertura.
- No hay integración continua configurada.

11.4 Escenarios de robustez

Ubicación: `escenarios/casos_robustos/` (10 archivos). Todos son escenarios **válidos**: superan `escenario/validador.py` y pueden cargarse con `cargar_escenario` . Su función es ejercer casos límite del cargador, del modelo de juego y de los algoritmos.

Archivo	Configuración verificable	Caso límite	Comportamiento esperado
`r01_meta_inalcanzable.json`	20×20; meta `(15,15)` encerrada por muros; A1 en `(2,2)`	Meta inalcanzable	BFS, DFS, UCS, A* y Beam Search terminan con `exito = False` sin bloquearse
`r02_inicio_igual_meta.json`	A1 y meta en `(5,5)`	Origen igual a meta	`exito = True` , longitud `0` , costo `0`
`r03_corredor_1x20.json`	Mapa 1×20; 10 celdas `camino` y 10 `pasto` ; A1 en `(0,0)` , meta `(0,19)`	Mapa no cuadrado	Camino único de 19 movimientos y costo `9×1 + 10×2 = 29`
`r04_unidad_encerrada.json`	A1 en `(5,5)` con muros en sus cuatro vecinos	Sin vecinos transitables	Las búsquedas fallan; en partida A1 solo dispone de `ESPERAR` (11 acciones legales para el bando A)
`r05_objetivo_en_muro.json`	Muro en `(10,10)` ; `prueba.objetivo = (10,10)`	Objetivo no transitable	La búsqueda se rechaza antes de ejecutar el algoritmo
`r06_costos_fraccionarios.json`	Terreno `autopista` con costo `0.5` en la fila 2; A1 en `(2,0)` , meta `(4,19)`	Costos fraccionarios	21 movimientos y costo `11.5` ; A* con Manhattan (escalada por `c_min = 0.5`) iguala a UCS
`r07_dimensiones_y_nombres_nuevos.json`	8×12; terrenos `sendero` , `arena` , `agua` y `roca` ; 4 unidades por bando; tipos de unidad y campos adicionales	Dimensiones, nombres y campos no previstos	El cargador ignora los campos extra y los algoritmos usan los costos del JSON
`r08_mapa_grande_40x40.json`	40×40 con terreno aleatorio de semilla fija y ~5 % de muros; A1 en `(2,2)` , meta `(37,37)`	Escala	Todos los algoritmos hallan solución; UCS y A* coinciden en costo
`r09_portador_en_su_base.json`	A1 en `(0,0)` con el recurso y `prueba.modo = "partida"`	Victoria en el estado inicial	`Reglas.ganador` devuelve `A` al cargar

Archivo	Configuración verificable	Caso límite	Comportamiento esperado
`r10_unidad_sobre_el_recurso.json`	A1 en `(10,10)`, celda del recurso, con `portador_recurso: null`	Recurso bajo una unidad al iniciar	El estado inicial marca a A1 como portadora; `ganador` es `None`

11.5 Escenarios inválidos

Ubicación: `escenarios/casos_invalidos/` (12 archivos). Cada archivo parte de un escenario base válido y rompe **una sola** regla de validación. El comportamiento esperado es que `cargar_escenario` lance `ErrorEscenario` y que `App.cargar_ruta` conserve intacto el escenario anterior (carga «todo o nada»).

Archivo	Regla incumplida	Error esperado (fragmento)
`inv01_dimensiones_inconsistentes.json`	`mapa.filas = 5` con un terreno de 4 filas	`mapa.filas es 5`
`inv02_terreno_inexistente.json`	El terreno usa `lava`, ausente de `tipos_terreno`	`no existe en tipos_terreno`
`inv03_costo_cero.json`	`pasto` transitable con costo `0`	`costo positivo`
`inv04_base_fuera_del_mapa.json`	Base A en la fila `9` de un mapa 4×4	`bases.A est`
`inv05_unidad_sobre_muro.json`	A1 ubicada en la celda `(1,1)`, que es muro	`no transitable`
`inv06_id_repetido.json`	A2 renombrada como `A1`	`repetido`
`inv07_bando_invalido.json`	Una unidad declara `bando = "C"`	`bando debe ser A o B`
`inv08_turno_invalido.json`	`turno = "Z"`	`turno debe ser A o B`
`inv09_portador_inexistente.json`	`portador_recurso = "X9"` sin unidad correspondiente	`no es una unidad existente`
`inv10_json_mal_formado.json`	JSON truncado (no parseable)	`JSON mal formado`
`inv11_falta_campo_obligatorio.json`	Se elimina la sección `bases`	`Faltan campos obligatorios`
`inv12_recurso_fuera_del_mapa.json`	Recurso en la columna `40` de un mapa 4×4	`recurso est`

12. Metodología experimental

12.1 Enfoque y diseño experimental

Para validar formalmente las propiedades algorítmicas del sistema, se adoptó un **enfoque metodológico empírico-cuantitativo y reproducible**. El diseño experimental evalúa sistemáticamente el comportamiento de los algoritmos de búsqueda no informada (BFS, DFS), basada en costos (UCS), informada (A* con heurísticas Manhattan, Propia y Euclidiana) y con memoria acotada (Beam Search) ante diversas configuraciones topológicas y de costos.

El objetivo de la experimentación es contrastar empíricamente las propiedades asintóticas teóricas frente al comportamiento real observado en tiempo de CPU, estados expandidos, memoria consumida y calidad de la solución encontrada.

12.2 Definición y clasificación de variables

El marco experimental clasifica rigurosamente las variables del sistema en tres categorías:

A. Variables Independientes (Factores manipulados)

Son las condiciones de entrada fijadas por el experimentador para someter a prueba los algoritmos:

1. **Algoritmo de búsqueda:** Algoritmo seleccionado para resolver la navegación ($extAlg \in \{\text{BFS}, \text{DFS}, \text{UCS}, \text{A}^*, \text{Beam Search}\}$).
2. **Función heurística en A* y Beam Search:** Función guía empleada ($h \in \{h_M\ (\text{Manhattan}), h_P\ (\text{Propia}), h_E\ (\text{Euclidiana})\}$).
3. **Ancho del haz (k) en Beam Search:** Capacidad de retención de la frontera por nivel ($k \in \{1, 2, 4, 8\}$).
4. **Topología de la cuadrícula y obstáculos:**

- *Espacio abierto:* Zona libre sin bloqueos intermedios (`caso01_bfs_vs_dfs.json`).
- *Rutas bifurcadas con costos heterogéneos:* Camino corto pero de alto costo vs. camino largo económico (`caso2_ruta_corta_vs_economica.json`).
- *Obstáculo cóncavo frontal:* Muro que bloquea la trayectoria euclidiana directa forzando desvío lateral (`caso3_obstaculo_y_heuristica.json`).
- *Distribución de costos graduada:* Zonas de pantano ($c = 7$), bosque ($c = 4$), pasto ($c = 2$) y camino ($c = 1$) (`caso4_comparacion_heuristicas.json`).
- *Corredores y trampas locales:* Laberinto para probar podas agresivas (`caso05_beam_search.json`).

5. **Dimensiones del mapa:** Escalas desde 4×4 (microescenarios) hasta 20×20 y 40×40 (gran escala).
6. **Algoritmo adversarial y su configuración** (casos 7–10): $\text{Alg}_{adv} \in \{\text{Minimax}, \text{Alfa-Beta}\}$, profundidad $d \in \{1, 2, 3, 4\}$ (la interfaz permite 1–8) y orden de acciones $\text{ord} \in \{\text{estandar}, \text{inverso}, \text{heurístico}\}$.

B. Variables Dependientes (Métricas de rendimiento observadas)

Parámetros cuantitativos medidos durante y después de la ejecución:

1. **Tiempo de ejecución neto (`tiempo_ms`):** Duración en milisegundos de la llamada algorítmica, medida con precisión de microsegundos mediante `time.perf_counter()` .
2. **Esfuerzo de búsqueda (Espacio de estados explorado):**
   - **Estados generados (N_{gen}):** Cantidad total de nodos introducidos a la estructura de frontera.
   - **Estados expandidos (N_{exp}):** Cantidad de nodos extraídos de la frontera y cuyos vecinos fueron evaluados.
3. **Consumo de memoria pico (`maximo_frontera`):** Tamaño máximo simultáneo alcanzado por la estructura de frontera (cola FIFO, pila, cola de prioridad o haz) a lo largo de la búsqueda.
4. **Calidad de la solución:**
   - **Longitud del camino (L):** Número de pasos o transiciones ($len(\text{camino}) - 1$).
   - **Costo total acumulado (C):** Suma aritmética de los costos de transición: $C = \sum_{i=1}^{L} \text{costo}(v_i)$.
5. **Validez formal del camino:** Evaluación booleana exhaustiva mediante el módulo `verificacion/verificador.py` (las 6 reglas de integridad espacial, ortogonalidad y costo).
6. **Métricas adversariales** (solo casos 7–10): **nodos evaluados** (hojas y estados sin acciones legales), **nodos generados** (hijos creados), **nodos podados** (hermanos descartados por corte $\alpha \geq \beta$) y **valor utilitario** V de la jugada elegida desde la perspectiva del bando en turno.

C. Variables de Control (Condiciones estandarizadas)

Factores mantenidos estrictamente constantes para garantizar validez interna y repetibilidad:

- **Entorno de ejecución:** Intérprete Python 3.12.0 sobre el mismo hardware de referencia en un único hilo de ejecución.
- **Orden de generación de sucesores:** Orden ortogonal determinista inalterable: $\text{ARRIBA} \rightarrow \text{ABAJO} \rightarrow \text{IZQUIERDA} \rightarrow \text{DERECHA}$.
- **Criterio de parada de meta:** Comprobación formal al extraer de la frontera (para UCS y A*) para asegurar consistencia teórica.
- **Estado inicial y objetivo:** Idénticas coordenadas de partida s_0 y meta g para todos los algoritmos comparados dentro de un mismo escenario.

12.3 Hipótesis experimentales formales

Se formularon siete hipótesis de trabajo para someter a prueba empírica:

- **Hipótesis 1 (Optimalidad de UCS vs. Ceguera de costo en BFS):**
  En escenarios con costos de terreno heterogéneos donde la ruta geométrica más corta atraviesa terrenos de alto costo, BFS hallará el camino con menor número de transiciones pero costo total subóptimo, mientras que UCS garantizará el costo óptimo a expensas de mayor número de pasos ($C_{\text{UCS}} < C_{\text{BFS}}$ y $L_{\text{UCS}} \geq L_{\text{BFS}}$).
- **Hipótesis 2 (Efectividad de la poda heurística en A* frente a búsquedas no informadas):**
  *Al disponer de una heurística admisible y consistente, el algoritmo A** guiará la frontera hacia el objetivo reduciendo drásticamente la cantidad de estados expandidos frente a UCS ($N_{\text{exp}}(A^*) \ll N_{\text{exp}}(\text{UCS})$), alcanzando exactamente el mismo costo óptimo ($C_{A^*} = C_{\text{UCS}}$).*
- **Hipótesis 3 (Dominancia y reducción de esfuerzo de h_P frente a h_M):**
  *Dado que la Heurística Propia satisface $h_P(s) \geq h_M(s)$ para todo estado no meta, A** con h_P expandirá un conjunto de nodos menor o igual al expandido con h_M ($N_{\text{exp}}(A^*_{h_P}) \leq N_{\text{exp}}(A^*_{h_M})$), haciéndose la reducción más pronunciada cuando el objetivo esté ubicado en un terreno de alto costo.*
- **Hipótesis 4 (Compromiso de memoria vs. completitud en Beam Search):**
  *Beam Search mantendrá el tamaño de su frontera estrictamente acotado por el parámetro k (memoria pico $= k$). Con valores reducidos de k ($k = 1, 2$) el algoritmo incurrirá en incompletitud (fallo ante obstáculos), mientras que al incrementar k recuperará la capacidad de encontrar el objetivo manteniendo un consumo de memoria órdenes de magnitud inferior a BFS y A**.

- **Hipótesis 5 (Ineficiencia patológica de DFS en grafos ortogonales):**
  En cuadrículas sin cotas de profundidad, DFS generará caminos con una longitud y costo desmesuradamente superiores a los óptimos ($L_{\text{DFS}} \gg L_{\text{BFS}}$) acumulando una cantidad elevada de nodos en su frontera en forma de pila.
- **Hipótesis 6 (Equivalencia con poda en búsqueda adversarial):**
  Para la misma utilidad y la misma profundidad, Alfa-Beta devolverá la acción y el valor idénticos a los de Minimax, evaluando estrictamente menos nodos ($N_{\text{eval}}(\text{AB}) < N_{\text{eval}}(\text{MM})$) y registrando nodos podados positivos.
- **Hipótesis 7 (Independencia del orden de acciones):**
  El orden de las acciones legales ($\text{estandar}, \text{inverso}, \text{heurístico}$) modifica el esfuerzo de exploración pero no el valor óptimo ni la acción seleccionada ($V_{\text{estandar}} = V_{\text{inverso}} = V_{\text{heurístico}}$).

12.4 Protocolo de control experimental y medición

Para asegurar que las mediciones temporales no se vean afectadas por fluctuaciones transitorias del sistema operativo, se estableció el siguiente protocolo:

1. **Aislamiento de procesos:** Las pruebas se ejecutaron cerrando aplicaciones en segundo plano no esenciales y fijando afinidad de proceso.
2. **Muestreo repetido:** Cada algoritmo se ejecuta $R = 5$ repeticiones consecutivas e independientes sobre la misma instancia del problema.
3. **Métrica temporal:** Se reporta la media aritmética del tiempo de CPU:

$$\bar{t} = \frac{1}{R} \sum_{r=1}^{R} t_r$$

4. **Validación automática:** Todo camino resultante es evaluado por `verificar_camino()`. Cualquier discrepancia de coordenadas o costo marcaría el resultado como inválido (`FALLO`); si el algoritmo no encontró ruta no hay nada que verificar y se registra `–`.
5. **Validación adversarial:** los casos 7–10 repiten 5 veces cada configuración y promedian `tiempo_ms`; la equivalencia Minimax/alfa-beta y la independencia del orden se contrastan en `tests/test_minimax.py` (Hipótesis 6 y 7).

13. Resultados cuantitativos y experimentales

13.1 Entorno y protocolo de experimentación

Los experimentos cuantitativos se ejecutaron de manera automatizada mediante el script reproducible `experimentos/ejecutar_experimentos_busqueda.py`, que genera `resultados_experimentos_busqueda.md` con tres secciones: **casos mínimos 1–5**, **casos robustos** `R01-R08` y **casos adversariales 7–10**.

- **Plataforma de pruebas:** Python 3.12.0 (mediciones originales, octubre de 2026) y **Python 3.14.0 sobre Windows 11 (x86_64) para la re-ejecución del 2026-10-20**, cuyos números son los que se reportan a continuación.
- **Protocolo de medición:** Cada configuración se evaluó con 5 repeticiones independientes sobre la misma instancia del problema para mitigar la varianza en el reloj del sistema operativo (`time.perf_counter`), reportando el tiempo promedio en milisegundos (`ms`). En los casos adversariales, nodos y tiempo provienen también del promedio de 5 corridas.
- **Verificación de correctitud:** Cada camino obtenido fue validado de forma automática e independiente por `verificar_camino()` comprobando origen, destino, límites, transitabilidad, ortogonalidad y consistencia exacta del costo acumulado. Si el algoritmo no encontró ruta, la columna **Verif.** registra `–` (no hay camino que validar); si el objetivo no es transitable, el caso se marca **Rechazado** antes de invocar al algoritmo.
- **Métricas recolectadas (búsqueda):**
  - **Éxito:** Booleano que indica si se alcanzó el estado objetivo.
  - **Longitud:** Número de movimientos o pasos efectivos ($len(\text{camino}) - 1$).
  - **Costo total:** Suma acumulada de los costos de transición del terreno.
  - **Estados generados:** Total de nodos insertados en la frontera de búsqueda.
  - **Estados expandidos:** Nodos extraídos de la frontera y cuyos sucesores fueron evaluados.
  - **Máximo en frontera:** Tamaño máximo simultáneo alcanzado por la estructura de frontera (memoria pico).
  - **Tiempo promedio (ms):** Tiempo de cómputo neto del algoritmo.
- **Métricas recolectadas (adversarial):** acción recomendada, valor utilitario, profundidad explorada, nodos generados, nodos evaluados (hojas), nodos podados y tiempo promedio.

13.2 Caso 1: BFS vs. DFS (Espacio abierto con pocos obstáculos)

- **Archivo de escenario:** `escenarios/casos minimos/caso01_bfs_vs_dfs.json`
- **Dimensiones:** Cuadrícula de 20×20 con terreno mayoritario `camino` ($c = 1$) y escasos obstáculos.
- **Problema evaluado:** Origen unidad A1 en $(4, 1)$ hacia objetivo en $(15, 16)$. Distancia Manhattan directa: 26 unidades.

Algoritmo / Configuración	Éxito	Longitud	Costo	Generados	Expandidos	Máx Frontera	Tiempo (ms)	Verificación
BFS	Sí	**27**	27.0	358	350	23	1.664	OK
DFS	Sí	235	235.0	592	309	**263**	2.185	OK
UCS	Sí	27	27.0	358	350	23	2.106	OK
A* (Manhattan)	Sí	**27**	27.0	**82**	**27**	55	**0.517**	OK
A* (Propia)	Sí	**27**	27.0	**82**	**27**	55	0.310	OK
Beam ($k = 2$)	Sí	27	27.0	107	53	**2**	0.329	OK

Análisis del Caso 1:

1. **Comportamiento de BFS:** Garantiza el camino óptimo en cantidad de pasos (27 movimientos). Al ser una búsqueda no informada en costo uniforme, expande radialmente en ondas (350 nodos expandidos).
2. **Comportamiento patológico de DFS:** Al descender ciegamente por la primera rama profunda disponible, genera un camino serpenteante y altamente redundante de **235 pasos** (casi 9 veces más largo que el óptimo). Su estructura de pila almacena hasta **263 nodos simultáneos** en frontera, evidenciando su ineficiencia en grafos con cuadrículas sin poda por profundidad.
3. **Eficacia de A*:** Al contar con costo homogéneo ($c = 1$), la distancia Manhattan es una cota muy precisa. A* expande **exactamente 27 nodos** (la línea óptima paso a paso) en solo 0.517 ms, ahorrando un **92.3% de expansiones** respecto a BFS.
4. **Comportamiento de Beam Search ($k = 2$):** En terreno abierto con gradiente heurístico continuo hacia el objetivo, un haz de apenas 2 estados encuentra la solución óptima utilizando solo **2 estados en memoria pico**.

13.3 Caso 2: Ruta corta vs. ruta económica (BFS vs. UCS)

- **Archivo de escenario:** `escenarios/casos minimos/caso2_ruta_corta_vs_economica.json`
- **Dimensiones:** 20×20 con terrenos heterogéneos: `camino` ($c = 1$), `pasto` ($c = 2$), `bosque` ($c = 4$), `pantano` ($c = 7$) y `muro` (no transitable).
- **Problema evaluado:** Origen unidad A1 en $(2, 2)$ hacia objetivo en $(2, 8)$. En línea recta intermedian celdas de pantano de alto costo; paralelamente existe una ruta de celdas tipo camino que rodea la zona.

Algoritmo / Configuración	Éxito	Longitud	Costo	Generados	Expandidos	Máx Frontera	Tiempo (ms)	Verificación
BFS	Sí	**6**	60.0	41	33	9	0.216	OK
DFS	Sí	142	288.0	315	162	153	0.921	OK
UCS	Sí	10	**19.0**	85	68	19	0.288	OK
A* (Manhattan)	Sí	10	**19.0**	45	32	14	0.165	OK
A* (Propia)	Sí	10	**19.0**	**19**	**11**	**8**	**0.070**	OK

Análisis del Caso 2:

1. **Divergencia entre longitud y costo:** Este caso evidencia el contraste fundamental entre algoritmos basados en profundidad vs. costo acumulado.
   - **BFS** optimiza exclusivamente la cantidad de transiciones: halla una ruta directa de **6 pasos**, pero al atravesar celdas de pantano ($c = 7$) incurre en un costo total de **60.0**.
   - **UCS** prioriza por costo acumulado: rodea el obstáculo de alto costo a través de terreno plano ($c = 1$ y $c = 2$), produciendo un camino de **10 pasos** pero con un costo óptimo de apenas **19.0** (ahorro económico del 68.3%).
2. **Desempeño de Heurísticas en A*:** Tanto Manhattan como Propia encuentran la solución de costo óptimo (19.0), pero **A* con Heurística Propia** expande solo **11 nodos** frente a los 68 de UCS y 32 de Manhattan, al tomar en cuenta el costo exacto de la celda de destino.

13.4 Caso 3: Obstáculo cóncavo y evasión con heurística

- **Archivo de escenario:** `escenarios/casos minimos/caso3_obstaculo_y_heuristica.json`
- **Dimensiones:** 20×20 con una barrera no transitable de muros interpuesta directamente entre el origen y el destino.
- **Problema evaluado:** Origen unidad A1 en $(10, 2)$ hacia objetivo en $(10, 12)$.

Algoritmo / Configuración	Éxito	Longitud	Costo	Generados	Expandidos	Máx Frontera	Tiempo (ms)	Verificación
BFS	Sí	18	36.0	300	280	20	0.959	OK
UCS	Sí	18	36.0	300	280	20	1.184	OK
A* (Manhattan)	Sí	18	36.0	223	185	40	**1.112**	OK
A* (Propia)	Sí	18	36.0	**206**	**167**	**39**	1.248	OK

Análisis del Caso 3:

1. **Evasión de mínimos locales:** Ante una barrera frontal de obstáculos, los algoritmos informados (A*) exploran inicialmente las celdas frontales del muro atraídos por la heurística Manhattan antes de rodearlo lateralmente.
2. **Preservación de optimalidad y ahorro:** A pesar de la concavidad, la consistencia de las heurísticas garantiza encontrar el camino óptimo (longitud 18, costo 36.0). A* con Heurística Propia logra el objetivo expandiendo **167 nodos frente a los 280 de UCS/BFS**, un ahorro superior al **40.3%**.

13.5 Caso 4: Comparación formal de heurísticas en A* (Manhattan vs. Propia)

- **Archivo de escenario:** `escenarios/casos minimos/caso4_comparacion_heuristicas.json`
- **Dimensiones:** 20×20 con distribución heterogénea de costos de terreno diseñada para evaluar el grado de información de la función heurística.
- **Problema evaluado:** Origen unidad A1 en $(10, 2)$ hacia objetivo en $(10, 16)$.

Algoritmo / Heurística	Éxito	Longitud	Costo	Generados	Expandidos	Máx Frontera	Tiempo (ms)	Verificación
UCS (Sin información, $h = 0$)	Sí	22	52.0	388	384	22	2.026	OK
A* (Heurística Manhattan)	Sí	22	52.0	372	353	43	3.062	OK
A* (Heurística Propia)	Sí	22	52.0	**273**	**232**	43	**2.218**	OK

Análisis del Caso 4 (Dominancia y poda):

1. **Igualdad de optimalidad:** Las tres búsquedas convergen exactamente en el mismo costo mínimo global (52.0) y misma longitud (22 pasos), ratificando la admisibilidad de ambas funciones heurísticas.
2. **Dominancia empírica demostrada:**
   - Heurística Manhattan: $h_M(s) = d_M(s, \text{meta}) \cdot c_{\min}$.
   - Heurística Propia: $h_P(s) = (d_M(s, \text{meta}) - 1) \cdot c_{\min} + \text{costo}(\text{meta})$.
   - Como $\text{costo}(\text{meta}) \geq c_{\min}$, se cumple matemáticamente que $\forall s, h_P(s) \geq h_M(s)$.
   - Consecuencia práctica observada: **A* con Heurística Propia expande solo 232 nodos frente a 353 con Manhattan**, logrando una **reducción del 34.3% en el espacio de búsqueda explorado** y un tiempo de cómputo 27.6% más veloz (2.218 ms frente a 3.062 ms en esta corrida).

13.6 Caso 5: Búsqueda con memoria acotada (Beam Search variando k)

- **Archivo de escenario:** `escenarios/casos minimos/caso05_beam_search.json`
- **Dimensiones:** 20×20 con corredores y obstáculos estructurados.
- **Problema evaluado:** Origen unidad A1 en $(17, 3)$ hacia objetivo en $(8, 17)$.
- **Algoritmos evaluados:** BFS y A* como líneas base de referencia vs. Beam Search con $k \in \{1, 2, 4, 8\}$.

Algoritmo / Configuración	Éxito	Longitud	Costo	Generados	Expandidos	Máx Frontera	Tiempo (ms)	Verificación
BFS (Línea base exhaustiva)	Sí	31	31.0	329	321	20	1.324	OK

Algoritmo / Configuración	Éxito	Longitud	Costo	Generados	Expandidos	Máx Frontera	Tiempo (ms)	Verificación
A* Manhattan (Línea base informada)	Sí	**31**	**31.0**	219	178	40	2.545	OK
Beam ($k = 1$, Greedy local)	**No**	—	—	25	9	**1**	0.206	—
Beam ($k = 2$)	**No**	—	—	31	17	**2**	0.286	—
Beam ($k = 4$)	Sí	55	55.0	266	213	**4**	3.037	OK
Beam ($k = 8$)	Sí	**31**	**31.0**	276	234	**8**	2.906	OK

Análisis del Caso 5 (Compromiso memoria vs. completitud y optimalidad):

1. **Incompletitud con haz estrecho ($k = 1$ y $k = 2$):** Al podar agresivamente todos los estados excepto los mejores k en cada nivel, el algoritmo descarta ramas alternativas y queda atrapado en un callejón sin salida (mínimo local), terminando con `exito = False`.
2. **Suboptimalidad con haz intermedio ($k = 4$):** Logra sortear el obstáculo pero al haber podado el camino principal más directo, encuentra una solución que requiere **55 pasos**, casi el doble de la ruta óptima.
3. **Equilibrio y eficiencia con $k = 8$:** Con un haz de 8 estados, Beam Search encuentra la **solución óptima exacta (31 pasos, costo 31.0)** manteniendo un consumo de memoria estrictamente acotado a **máximo 8 estados en frontera**, en comparación con los 40 estados simultáneos que requiere A* y los 20 de BFS.

13.7 Casos robustos de búsqueda (`R01` a `R08`)

- **Archivo de resultados completo:** `resultados_experimentos_busqueda.md`, sección «2. Casos robustos de búsqueda».
- **Objetivo:** someter a los algoritmos a situaciones límite del cargador y del mapa (metas inalcanzables, origen igual a meta, mapas no cuadrados, unidades encerradas, objetivos no transitables, costos fraccionarios, nombres de terreno nuevos y mapas grandes), sin variar los algoritmos.

Robusto	Situación límite	Resultado observado
`R01` meta inalcanzable	Meta rodeada de muros	Los cinco algoritmos terminan con `exito = No` (BFS/UCS/A* expanden los 391 estados transitables; DFS usan pila de 348). **Ninguno se bloquea**: se respeta la completitud de la búsqueda con detección de exhaustión.
`R02` origen = meta	$s_0 = g$	Camino vacío: longitud `0`, costo `0.0`, 1 estado generado, 0 expandidos, verificación `OK`.
`R03` corredor 1×20	Mapa no cuadrado, camino único	Todos hallan 19 movimientos con costo **29.0** ($9 \times 1 + 10 \times 2$), coincidiendo con el cálculo previo de la sección 11.4; Beam con $k = 1$ también (sin bifurcaciones que podar).
`R04` unidad encerrada	A1 rodeada de muros	1 estado generado y 1 expandido por algoritmo: la búsqueda se agota de inmediato con `exito = No` sin recorrer el mapa.
`R05` objetivo en muro	`prueba.objetivo` sobre celda no transitable	**Rechazado antes de invocar al algoritmo** (misma precondición que `App._problema`); no se ejecuta ninguna búsqueda.
`R06` costos fraccionarios	Terreno `autopista` con $c = 0.5$	BFS devuelve costo `21.0` (ignora costos) y DFS `357.5` en 367 movimientos; **UCS y ambas A* convergen en `11.5`**; A* con Heurística Propia expande solo 21 nodos (frente a 189 de UCS).
`R07` dimensiones y nombres nuevos	8×12, terrenos `sendero` / `arena` / `agua` / `roca`	El cargador acepta los nombres declarados en el JSON y los algoritmos usan sus costos: UCS y A* encuentran `15.0` frente a `25.0` de BFS.
`R08` mapa 40×40	Escala con ~5% de muros aleatorios (semilla fija)	Óptimo de 70 movimientos y costo `77.0` (UCS y A* coinciden); BFS devuelve `142.0` y DFS `2028.0` con 966 movimientos y 1220 nodos en pila. **A* expande 694 nodos frente a los 1509 de UCS (54.1% menos)**.

Análisis del conjunto robusto:

1. **Completitud y terminación:** los casos sin solución (`R01` , `R04`) terminan de forma limpieta en todos los algoritmos, incluido Beam Search con haz acotado.
2. **Degeneraciones bien manejadas:** `R02` (camino nulo) y `R05` (rechazo previo) no producen errores ni excepciones.
3. **Consistencia de costos:** en los escenarios con costos uniformes o fraccionarios, UCS y A* coinciden en el costo óptimo en todos los casos, lo que confirma la admisibilidad de las heurísticas incluso con $c_{\min} = 0.5$.
4. **Comportamiento esperado de DFS:** en `R06` y `R08` DFS sigue siendo el peor tanto en costo como en memoria (362 y 1220 nodos de pila respectivamente), en línea con la Hipótesis 5.

13.8 Casos adversariales (7 al 10): Minimax frente a Poda Alfa-Beta

- **Archivos:** `escenarios/casos minimos/caso07_decision_adversarial.json` , `caso08_minimax_vs_alfa_beta.json` , `caso09_orden_acciones.json` , `caso10_profundidad.json` .
- **Archivo de resultados completo:** `resultados_experimentos_busqueda.md` , sección «3. Casos adversariales» (promedios de 5 corridas).
- **Todas las configuraciones parten del bando A en turno**; la utilidad se reporta desde su perspectiva.

Caso 7: Decisión adversarial (interceptación táctica)

Algoritmo / Config	Acción recomendada	Valor	Prof.	Generados	Evaluados	Podados	Tiempo (ms)
MINIMAX (prof 3, estandar)	A1 → (8, 9)	87.00	3	3234	3009	0	32.710
ALFA_BETA (prof 3, estandar)	A1 → (8, 9)	87.00	3	358	300	488	3.587

Análisis: ambos algoritmos eligen la misma jugada (interceptar al portador rival `B1`), valor `87.00` . La poda reduce las evaluaciones de **3009 a 300 (−90.0%)** y el tiempo de **32.710 ms a 3.587 ms (−89.0%)**, dejando en el árbol 488 ramas hermanas sin visitar. La prueba `test_caso7_intercepta_portador_adyacente` además comprueba que, tras la jugada, `B1` desaparece y `A1` pasa a ser portadora.

Caso 8: Minimax frente a poda alfa-beta

Algoritmo / Config	Acción recomendada	Valor	Prof.	Generados	Evaluados	Podados	Tiempo (ms)
MINIMAX (prof 3, estandar)	A1 → (9, 7)	5.00	3	3496	3255	0	33.834
ALFA_BETA (prof 3, estandar)	A1 → (9, 7)	5.00	3	628	543	600	6.169

Análisis (validación de la Hipótesis 6): misma acción y mismo valor (`5.00`), con **83.3% menos de nodos evaluados** (543 frente a 3255), **600 nodos podados** y un tiempo **81.8% menor**. Minimax, al no podar, registra `nodos_podados = 0` .

Caso 9: Impacto del orden de acciones en la poda alfa-beta

Orden de acciones	Acción recomendada	Valor	Prof.	Generados	Evaluados	Podados	Tiempo (ms)
`estandar`	A1 → (9, 7)	5.00	3	628	543	600	7.290
`inverso`	A1 → (9, 7)	5.00	3	2008	1829	622	42.203
`heurístico`	A1 → (9, 7)	5.00	3	276	231	392	10.021

Análisis (validación de la Hipótesis 7): los tres órdenes devuelven **exactamente el mismo valor y la misma acción**, pero el esfuerzo varía más de **8×** (de 231 a 1829 evaluaciones). El orden inverso es el peor (retrocede en la frontera buena) y el heurístico el mejor en nodos; no obstante su tiempo (10.021 ms) supera ligeramente al `estandar` (7.290 ms), porque ordenar exige evaluar una vez la utilidad de cada sucesor antes de explorar.

Caso 10: Efecto de la profundidad en Minimax / alfa-beta

Profundidad	Acción recomendada	Valor	Prof.	Generados	Evaluados	Podados	Tiempo (ms)
1	A1 → (9, 7)	5.00	1	16	15	0	0.328
2	A1 → (9, 7)	0.00	2	45	29	196	2.543
3	A1 → (9, 7)	5.00	3	276	231	392	8.264
4	A1 → (9, 7)	0.00	4	709	433	3416	39.421

Análisis: el coste crece rápidamente con d (de 15 a 433 evaluaciones y de 0.328 a 39.421 ms al pasar de 1 a 4 plies), mientras la poda acumula 3416 ramas descartadas en $d = 4$. El valor **alterna** `5.00 → 0.00 → 5.00 → 0.00` : con horizonte par, la ventana de respuesta del rival queda en un estado cuya utilidad evaluada es 0.00, mientras que con horizonte impar termina en un estado favorable. La acción elegida permanece estable (`A1 → (9,7)`), lo que muestra que la decisión no es sensible a la profundidad en este estado, aunque el valor sí lo es.

Síntesis de los casos adversariales:

- **Hipótesis 6 confirmada** (casos 7 y 8): alfa-beta reproduce el valor y la acción de Minimax con una fracción del trabajo.
- **Hipótesis 7 confirmada** (caso 9): el orden modifica el esfuerzo, no el resultado.
- La profundidad es la palanca de costo dominante (caso 10): duplicar d multiplica las evaluaciones por entre 3× y 10×.

14. Análisis teórico y empírico de complejidad

14.1 Parámetros formales del espacio de búsqueda

En el contexto de la cuadrícula 2D ortogonal y las reglas de TacticalGrid, los parámetros que gobiernan la complejidad asintótica son:

- b **(Factor de ramificación efectivo):** Número de acciones legales aplicables desde un estado. En cuadrícula con 4 direcciones ortogonales, $b \leq 4$. Puesto que un algoritmo no reinserta inmediatamente la casilla de la que proviene (evita retroceso trivial), el factor de ramificación hacia adelante satisface $b \leq 3$, reduciéndose aún más en presencia de bordes y muros.
- d **(Profundidad de la solución):** Longitud en número de pasos o transiciones del camino óptimo más superficial hacia el objetivo.
- m **(Profundidad máxima del espacio):** Longitud máxima admisible de un camino simple (sin ciclos) en la cuadrícula ($m \leq |S| \leq F \times C$).
- C^* **(Costo óptimo):** Costo total acumulado del camino óptimo hacia la meta.
- $\epsilon = c_{\min} > 0$**:** Cota inferior del costo de cualquier transición transitable en el escenario.
- k **(Ancho del haz en Beam Search):** Número máximo de estados candidatos retenidos en la frontera en cada nivel de profundidad.
- d_{adv} **(Profundidad adversarial):** Límite máximo del árbol de juego en Minimax y Alfa-Beta.

14.2 Tabla comparativa de complejidad asintótica

Algoritmo	Complejidad Temporal (Peor Caso)	Complejidad Espacial (Memoria Pico)	¿Completo?	¿Óptimo?
BFS (Búsqueda en Anchura)	$O(b^d)$	$O(b^d)$	Sí (si $b < \infty$)	Sí en cantidad de pasos; **No** en costos heterogéneos
DFS (Búsqueda en Profundidad)	$O(b^m)$	$O(b \cdot m)$ en árbol / $O(	V	)$ en grafo
UCS (Costo Uniforme / Dijkstra)	$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$	$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$	Sí (dado que $\epsilon = c_{\min} > 0$)	**Sí** (garantiza costo acumulado mínimo)
A* (Búsqueda Informada)	$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$ peor caso / $O(b^{\alpha d})$ con $\alpha < 1$	$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$ peor caso / $O(b^{\alpha d})$	Sí (con h admisible y $\epsilon > 0$)	**Sí** (con h admisible en árbol o consistente en grafo)
Beam Search (Haz con memoria k)	$O(k \cdot b \cdot d)$	$O(k \cdot d)$ en historial / $O(k)$ en frontera	**No** (puede podar ramas viables y quedar atrapado)	**No** (aproximación heurística subóptima)
Minimax (Adversarial puro)	$O(b^{d_{\text{adv}}})$	$O(b \cdot d_{\text{adv}})$	Sí (en árboles de juego finitos)	**Sí** (frente a rival óptimo en juego de suma cero)
Poda Alfa-Beta (Adversarial)	$O\left(b^{d_{\text{adv}}/2}\right)$ (orden óptimo)	$O(b \cdot d_{\text{adv}})$	Sí (idéntica decisión que Minimax)	**Sí** (misma solución que Minimax)

14.3 Justificación teórica detallada por algoritmo

1. Búsqueda en Anchura (BFS)

- **Complejidad Temporal:** BFS expande exhaustivamente todos los nodos de profundidad $0, 1, 2, \ldots, d$. El número total de nodos expandidos es la suma de la serie geométrica:

$$\sum_{i=0}^{d} b^i = \frac{b^{d+1} - 1}{b - 1} \in O(b^d)$$

- **Complejidad Espacial:** La frontera FIFO almacena simultáneamente todos los nodos del nivel más profundo d, requiriendo $O(b^d)$ memoria. Esta demanda exponencial hace que la memoria RAM sea el cuello de botella insalvable de BFS en mapas extensos.
- **Optimalidad:** Es óptimo únicamente cuando todos los pasos tienen el mismo costo ($c = 1$). Si los terrenos tienen costos variables, BFS ignora el costo acumulado y produce soluciones económicamente ineficientes (evidenciado en el Caso 2, donde costó 60.0 frente al óptimo de 19.0).

2. Búsqueda en Profundidad (DFS)

- **Complejidad Temporal:** Al descender sin control de costo, puede recorrer ramas profundas completas antes de encontrar la meta, incurriendo en un tiempo de $O(b^m)$ en el peor caso, donde m puede ser el número total de celdas transitables del mapa.
- **Complejidad Espacial:** En búsqueda sobre árbol retiene únicamente la rama actual, requiriendo $O(b \cdot m)$ de memoria. Sin embargo, en búsqueda sobre grafo (como en TacticalGrid) se almacena un conjunto de visitados para evitar ciclos infinitos, ocupando $O(|V|)$ en memoria.
- **Patología en Cuadrículas:** Como se constató en el Caso 1, DFS genera trayectorias desmesuradamente largas (235 pasos vs. 27 de BFS) y acumula cientos de estados en la pila (263 nodos en frontera pico).

3. Búsqueda de Costo Uniforme (UCS / Dijkstra)

- **Fundamento:** En lugar de expandir por profundidad, UCS expande contornos concéntricos de costo acumulado uniforme $g(n)$.
- **Complejidad Temporal y Espacial:** Siendo C^* el costo de la solución óptima y $\epsilon = c_{\min} > 0$ la cota inferior de costo por paso, cualquier camino que alcance el costo C^* consta de a lo sumo $\lfloor C^*/\epsilon \rfloor$ pasos.
  En el peor caso (por ejemplo, si existen celdas de costo casi nulo o cuando el objetivo se encuentra en el límite del contorno), el algoritmo debe explorar nodos hasta un contorno efectivo de profundidad $1 + \lfloor C^*/\epsilon \rfloor$.
  Por ende, el tiempo y la memoria de la cola de prioridad (`heapq`) están acotados por:

$$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$$

  Dado que $\epsilon = c_{\min} > 0$, la completitud y la optimalidad están matemáticamente garantizadas.

4. Algoritmo A* y Efecto del Error Heurístico $|h(n) - h^*(n)|$

- **Peor Caso:** Si la heurística no aporta información ($h(n) = 0$), A* se reduce idénticamente a UCS con complejidad $O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$.
- **Mejor Caso:** Si la heurística es perfecta ($h(n) = h^*(n)$), A* no expande ningún nodo fuera del camino óptimo, reduciendo la complejidad a un tiempo estrictamente lineal $O(d)$.
- **Caso Práctico y Error Heurístico:**
  Para heurísticas admisibles y consistentes en cuadrículas (h_M y h_P), el esfuerzo computacional depende directamente del **error heurístico absoluto**:

$$\epsilon_h(n) = |h^*(n) - h(n)|$$

  - Según el análisis clásico de Pearl (1984) y Russell & Norvig (2020), si el error heurístico absoluto satisface $|h^*(n) - h(n)| \leq O(\log h^*(n))$, la complejidad temporal de A* se reduce a **tiempo polinomial** en d.
  - Si el error heurístico crece linealmente con la distancia ($|h^*(n) - h(n)| \leq c \cdot h^*(n)$), el factor de ramificación efectivo se reduce de b a una constante estrictamente menor $b^* < b$, con complejidad $O((b^*)^d) = O(b^{\alpha d})$ con $\alpha < 1$.
  - **Dominancia y Poda:** Al cumplirse que $\forall s, h_P(s) \geq h_M(s)$, el error heurístico de la Heurística Propia es menor o igual al de Manhattan:

$$\epsilon_{h_P}(s) \leq \epsilon_{h_M}(s)$$

    Esto comprime el volumen de nodos con $f(n) \leq C^*$, lo que explica por qué en el Caso 4 la Heurística Propia redujo las expansiones de 353 a 232 estados (ahorro neto del 34.3%).

5. Búsqueda con Memoria Acotada (Beam Search con parámetro k)

- **Fundamento:** Variante de búsqueda en haz que limita estrictamente el tamaño de la frontera a un máximo de k estados en cada nivel de profundidad.
- **Complejidad Temporal:** En cada nivel se expanden a lo sumo k estados. Cada estado produce a lo sumo b sucesores, generando un conjunto de a lo sumo $k \cdot b$ candidatos. Filtrar los k mejores estados mediante ordenamiento o selección toma $O(k \cdot b \log k)$.
  A lo largo de los d niveles de profundidad hasta el objetivo, la complejidad temporal total es:

$$O(k \cdot b \cdot d)$$

  A diferencia de BFS o UCS cuyo tiempo crece exponencialmente ($O(b^d)$), el tiempo de Beam Search es **lineal respecto a la profundidad** d.
- **Complejidad Espacial:** El consumo de memoria en la frontera está estrictamente acotado por $O(k)$ estados simultáneos (o $O(k \cdot d)$ si se retiene el historial de padres para reconstruir la ruta).
- **Compromiso Incompletitud / Suboptimalidad:** Como se demostró experimentalmente en el Caso 5:
  - Con $k = 1$ y $k = 2$, el haz es excesivamente estrecho y queda atrapado en callejones sin salida (incompleto).
  - Con $k = 4$, halla una solución pero subóptima (55 pasos vs. 31 óptimos).
  - Con $k = 8$, alcanza la solución óptima exacta consumiendo una memoria pico de solo 8 estados.

6. Búsqueda Adversarial (Minimax y Poda Alfa-Beta) — [Fundamento Teórico]

- **Minimax puro:** Realiza un recorrido en profundidad (DFS retroactivo) sobre el árbol de juego por turnos. Explora todos los nodos hasta la profundidad límite d_{adv}.
  - Tiempo: $O(b^{d_{\text{adv}}})$.
  - Espacio: $O(b \cdot d_{\text{adv}})$ (memoria lineal en la profundidad).
- **Poda Alfa-Beta:** Mantiene los valores α (mejor opción para MAX) y β (mejor opción para MIN). Si en un subárbol se detecta que el rival puede forzar un resultado peor que una opción ya descubierta, se poda inmediatamente la rama.
  - Con un ordenamiento óptimo de acciones (evaluando primero las mejores jugadas), el factor de ramificación efectivo se reduce a la raíz cuadrada $\sqrt{b}$, alcanzando una complejidad temporal de:

$$O\left(b^{d_{\text{adv}}/2}\right)$$

  - Esta reducción cuadrática en el tiempo permite que Alfa-Beta analice el **doble de profundidad** táctica que Minimax bajo el mismo límite de tiempo.
  - **Función de evaluación implementada** (`algoritmos/minimax._utilidad`): valores terminales ± 100000 (según `Reglas.ganador`), más material (35 por unidad de diferencia), más posesión del recurso ($\pm(180 - 8 \cdot d)$ respecto a la base) con bono/penalización de presencia rival cercana ($\max(0, 5 - d) \cdot 12$), o, si el recurso está en el suelo, $5 \cdot (d_{\text{rival}} - d_{\text{propio}})$.
  - **Evidencia cuantitativa** (sección 13.8): en el Caso 8, alfa-beta reproduce exactamente la acción y el valor de Minimax (5.00) evaluando **543 nodos frente a 3255** (−83.3%) con 600 podas; en el Caso 9, la variación del orden de acciones mantiene el valor y multiplica el esfuerzo por hasta 8×.

14.4 Contraste empírico: Teoría vs. Mediciones Reales en TacticalGrid

Las predicciones asintóticas se contrastan directamente con las mediciones empíricas recolectadas en la Sección 13:

Fenómeno Teórico	Predicción Asintótica	Evidencia Empírica en TacticalGrid
Ineficiencia de DFS en cuadrícula	$O(b^m)$ tiempo y $O(	V
Ceguera de costo en BFS	Optimiza d, no C^*	**Caso 2:** BFS halló la ruta más corta (6 pasos) pero con costo excesivo de 60.0; UCS halló el costo óptimo de 19.0 en 10 pasos.
Poda de A* vs. UCS	$E(A^*) \ll E(\text{UCS})$ por guía de $h(n)$	**Caso 1:** A* expandió exactamente 27 nodos frente a los 350 nodos de UCS (reducción del 92.3% del árbol explorado).
Dominancia $h_P \geq h_M$	$E(A^*_{h_P}) \subseteq E(A^*_{h_M})$	**Caso 4:** A* con Heurística Propia expandió 232 nodos frente a 353 con Manhattan (ahorro de 121 estados).

Fenómeno Teórico	Predicción Asintótica	Evidencia Empírica en TacticalGrid
Memoria acotada de Beam Search	Máximo de frontera acotado estrictamente por k	**Caso 5:** con $k = 8$, Beam Search resolvió la ruta óptima de 31 pasos manteniendo una frontera pico de exactamente 8 estados, frente a 40 de A* y 20 de BFS.
Incompletitud de Beam con haz estrecho	Poda de ramas viables → puede fallar	**Caso 5:** $k = 1$ y $k = 2$ terminan con `exito = No` ; $k = 4$ halla una ruta subóptima (55 pasos); $k = 8$ recupera el óptimo.
Equivalencia con poda alfa-beta	$V(\mathrm{AB}) = V(\mathrm{MM})$ con menos nodos	**Casos 7 y 8:** misma acción y valor; 300 vs 3009 y 543 vs 3255 nodos evaluados (−90.0% y −83.3%).
Efecto del orden en la poda	Hasta $O(b^{d/2})$ con orden óptimo	**Caso 9:** mismo valor con los tres órdenes; evaluaciones de 231 (heurístico) a 1829 (inverso).

15. Limitaciones técnicas, deuda técnica y funcionalidades pendientes

15.1 Funcionalidades pendientes

Elemento	Evidencia	Estado
Beam Search	`algoritmos/beam_search.py` registra `BEAM_SEARCH` ; `tests/test_beam_search.py` y Caso 5	**Implementado y probado** (`k = 1, 2, 4, 8`)
Comparación con `k`	Caso 5 en §13.6 y en `resultados_experimentos_busqueda.md`	**Registrado** para los cuatro valores exigidos
Minimax	`algoritmos/minimax.py` , interfaz y `tests/test_minimax.py`	Implementado y probado en los escenarios 7–10
Alfa-beta	`algoritmos/minimax.py` , interfaz y reporte experimental	Implementado y probado en los escenarios 7–10
Controlador de partida por IA	`turno_ia` admite Humano, Azar, Minimax y Alfa-Beta	Implementado
Tabla «esperado vs. obtenido»	`experimentos/ejecutar_casos.py` + `esperados.json`	**Pendiente**: el script no ejecuta (véase 15.2)

15.2 Deuda técnica y limitaciones verificables

Aspecto	Observación
Ausencia de archivo de dependencias	No hay `requirements.txt` ni equivalente; no se fijan versiones
Ausencia de empaquetado	No hay `__init__.py` en `escenario` , `juego` , `interfaz` , `verificacion` , `tests` ni `experimentos` ; se depende de paquetes de espacio de nombres
Sin linter/formateador/CI	No se encontró configuración
Docstrings con validación incompleta	Varios métodos contienen `Student validation: (completar) — [COMPLETAR CUANDO SE IMPLEMENTE]`
Campo `tipo` de unidades	Se acepta en el JSON pero no se modela en `Unidad` ni se usa en reglas
`version` sin validación de valor	Solo se exige la clave
Bases y recurso sobre obstáculos	No se rechazan (el PDF solo prohíbe unidades sobre celdas no transitables); el validador emite un **aviso** no bloqueante (`advertencias_datos`)
Generador de casos	`generar_casos_minimos.py` solo genera los casos 1, 5 y 6 (el resto se editó a mano) y escribe en `escenarios/casos_minimos` (guion bajo), no en `casos minimos`
Generador de robustos/inválidos	`generar_casos_robustez.py` escribe en `escenarios/robustez` y `escenarios/invalidos` , carpetas distintas de las versionadas `casos_robustos` y `casos_invalidos`

Aspecto	Observación
`experimentos/ejecutar_casos.py` no ejecuta	Importa `algoritmos.adversarial` y `verificacion.verificador_adversarial`, módulos inexistentes (`ModuleNotFoundError: No module named 'algoritmos.adversarial'`)
`esperados.json` fuera de sincronía	Sus claves de configuración usan `orden`, mientras que `minimax.py`/la interfaz usan `orden_acciones`; además los algoritmos adversariales declarados ahí esperan el módulo `algoritmos.adversarial` ausente
`resultados/tabla_esperado_vs_obtenido.*` desactualizada	Todas las filas dicen `DIFIERE` porque apuntan a `casos_minimos/...` inexistentes (corrida previa de `ejecutar_casos.py`)
Desplegable de heurísticas de la interfaz	`HEURISTICAS = ("manhattan", "propia")` en `interfaz/app.py`: la heurística `euclidiana` solo es accesible por código
Bytecode versionado	`__pycache__/*.pyc` están rastreados en Git pese a aparecer en `.gitignore` (se añadieron antes de la regla), por lo que aparecen como modificados en cada corrida
Árbol de trabajo sin commitear (2026-10-20)	`M tests/test_beam_search.py`, `M tests/test_paso1_2.py`, `M experimentos/ejecutar_experimentos_busqueda.py`, `M documentacion/*`; `?? experimentos/ejecutar_casos.py`, `?? experimentos/esperados.json`, `?? experimentos/generar_casos_robustez.py`, `?? resultados/`
Crecimiento del historial	El historial se mantiene en memoria y crece con cada ejecución; cada ejecución de A* agrega una entrada por heurística ejecutada
Costo en BFS/DFS	El costo reportado corresponde al camino hallado; no se optimiza por costo
Condiciones previas no verificadas en tiempo de ejecución	Varias funciones asumen precondiciones (`Reglas.aplicar`, `reconstruir_camino`, `Mapa.costo`) y pueden lanzar excepciones si se violan
Mediciones de tiempo	`tiempo_ms` mide solo la llamada al algoritmo, no la verificación ni la interfaz
Escenario `caso05_beam_search.json`	Ejecutable; k = 1 y 2 no hallan ruta, k = 4 y 8 sí (ver informe técnico)
Rutas absolutas en el manual	Las rutas citadas son relativas a la raíz del repositorio
Heurística euclidiana	Presente en `a_estrella.py` como tercera heurística de comparación

15.3 Aspectos no verificados

- No se verificó el comportamiento visual de la interfaz en ejecución (no se abrió la ventana durante esta elaboración).
- No se verificó el rendimiento en escenarios de gran tamaño más allá de lo que miden las pruebas.
- No se verificó la ejecución de `experimentos/generar_casos_minimos.py` ni `experimentos/generar_casos_robustez.py` (regenerarían carpetas con otros nombres) ni `experimentos/ejecutar_casos.py` (no ejecuta).
- No se verificó compatibilidad real con Python 3.9, 3.10 ni 3.11; se ejecutó con 3.12.0 (versión inicial) y 3.14.0 (re-ejecución del 2026-10-20).
- No se pudo determinar la versión del proyecto: no hay etiquetas ni metadatos.
- No se verificó el contenido de `heuristicas/` porque está vacío; `docs/` ya no existe en el repositorio.

16. Reporte del uso de inteligencia artificial

16.1 Declaración de uso

El equipo de TacticalGrid utilizó herramientas de inteligencia artificial generativa como apoyo al proceso de desarrollo, documentación y experimentación. El presente reporte describe de manera transparente qué se delegó a la IA, cómo se validó y cuál fue la contribución neta del equipo estudiantil, en cumplimiento de las normas de integridad académica y el modelo de uso responsable de IA establecido en el curso.

> **Postura del equipo:** La IA fue empleada como herramienta de asistencia, nunca como sustituto del entendimiento conceptual. Toda propuesta generada por IA fue revisada, probada y, cuando fue necesario, corregida por el equipo antes de ser incorporada al proyecto.

16.2 Herramientas de IA utilizadas

Herramienta	Modalidad de uso	Áreas de aplicación principales
Antigravity (Agente de codificación avanzado — Google DeepMind)	Asistente conversacional con capacidad de leer, escribir y ejecutar código en el repositorio	Generación de código base, documentación técnica, experimentos y análisis
Modelos de lenguaje de Google (Gemini)	Consultas de diseño, revisión de lógica algorítmica	Revisión de estructuras de datos, aclaraciones conceptuales

16.3 Desglose por área de soporte

A. Diseño de la arquitectura y estructura del proyecto

- **Intervención de IA:** Se consultó sobre la separación en capas (escenario, juego, algoritmos, interfaz, verificación) y el uso del patrón de diseño **Plugin/Registro** (`@registrar`) para desacoplar los algoritmos de la interfaz.
- **Validación humana:** El equipo evaluó la idoneidad de la arquitectura en relación con los requerimientos del proyecto, ajustó las responsabilidades de cada módulo y verificó que no existieran dependencias circulares ejecutando las pruebas unitarias.

B. Implementación de algoritmos de búsqueda

- **Intervención de IA:** La IA generó los esqueletos iniciales de BFS, DFS, UCS y A*, incluyendo la estructura de la cola de prioridad (`heapq`), el diccionario `mejor_g` para evitar reexpansiones y el mecanismo de desempate por valor heurístico y contador de inserción.
- **Código afectado:** `algoritmos/busqueda_no_informada.py` , `algoritmos/busqueda_con_costo.py` , `algoritmos/a_estrella.py` , `algoritmos/beam_search.py` .
- **Validación humana:** Cada algoritmo fue sometido a la batería de pruebas unitarias en `tests/` . Se verificó manualmente que:
  i. BFS y DFS no usaran costos para seleccionar el camino.
  ii. UCS produjera el costo acumulado mínimo en todos los escenarios de prueba con costos heterogéneos.
  iii. A* con heurística admisible encontrara el mismo costo que UCS.
  iv. Beam Search devolviera `exito = False` al quedarse sin candidatos en el haz.

C. Diseño e implementación de las heurísticas

- **Intervención de IA:** La IA propuso el diseño de `h_manhattan` (multiplicada por `costo_minimo()` para garantizar admisibilidad) y la formulación inicial de `h_propia` como `(d - 1) * c_min + costo(objetivo)` .
- **Validación humana:** El equipo verificó matemáticamente la admisibilidad, la consistencia y la dominancia de ambas heurísticas, ejecutó la prueba `test_admisibilidad_heuristicas` y comparó los resultados con los casos de prueba `caso3` y `caso4` , confirmando la reducción de nodos expandidos prevista teóricamente.

D. Diseño del modelo de estados y las reglas del juego

- **Intervención de IA:** Se consultó sobre el uso de `@dataclass(frozen=True)` para la inmutabilidad de `Unidad` , `Estado` y `Accion` , y la canonicidad de ordenar las unidades por `id` para garantizar la hashabilidad del estado y la correctitud del test de igualdad.
- **Código afectado:** `juego/estado.py` , `juego/reglas.py` .
- **Validación humana:** La prueba `test_hashable_e_inmutable` verificó automáticamente que dos estados iguales tuvieran el mismo hash. Las pruebas de `test_acciones_y_sucesor` , `test_recoger_recurso` , `test_interceptar` y `test_victoria` verificaron la correctitud de las reglas R1–R7 en escenarios controlados.

E. Interfaz gráfica con Tkinter

- **Intervención de IA:** La IA generó la estructura de la ventana principal con sistema de pestañas (`ttk.Notebook`), el widget de dibujo del tablero (`Tablero` como subclase de `tk.Canvas`), la paleta de colores oscura y el mecanismo de capas visuales (camino, explorados, inicio, objetivo, selección).
- **Código afectado:** `interfaz/app.py` , `interfaz/tablero.py` , `interfaz/graficas.py` .
- **Validación humana:** El equipo ejecutó la aplicación completa, verificando visualmente el comportamiento de cada pestaña, la carga de todos los escenarios de prueba, la visualización del camino y la comparación de algoritmos.

F. Generación de escenarios de prueba JSON

- **Intervención de IA:** La IA colaboró en el diseño de los 12 escenarios inválidos, los 10 escenarios robustos y los 8 casos mínimos, asegurando que cada uno ejerciera exactamente una condición límite diferente.
- **Validación humana:** El equipo ejecutó `cargar_escenario()` sobre cada archivo, verificando que los escenarios inválidos fueran rechazados con el mensaje de error esperado y que los válidos cargaran correctamente.

G. Documentación técnica

- **Intervención de IA:** El agente Antigravity generó y actualizó el manual técnico a partir del análisis directo del código fuente, incluyendo las secciones de formalización matemática, demostración de heurísticas, análisis de complejidad, resultados cuantitativos y metodología experimental.
- **Validación humana:** El equipo revisó el contenido de cada sección, corrigió imprecisiones factuales (números de línea, estados de implementación), integró datos empíricos reales ejecutando los experimentos y aprobó el lenguaje y nivel de formalización antes de su inclusión definitiva.

H. Script de experimentación automatizada

- **Intervención de IA:** Se generó el script `experimentos/ejecutar_experimentos_busqueda.py` que automatiza la ejecución de los casos de prueba mínimos, robustos y adversariales, registra métricas con 5 repeticiones y valida los caminos resultantes.
- **Validación humana:** El equipo ejecutó el script y revisó que las métricas reportadas fueran coherentes con las esperadas según la teoría (por ejemplo, que A* con `h_propia` expandiera menos nodos que con `manhattan`, o que alfa-beta evaluara menos nodos que Minimax).

I. Búsqueda adversarial (Minimax y poda alfa-beta)

- **Intervención de IA:** La IA propuso la estructura recursiva compartida `_buscar(...)` de `algoritmos/minimax.py`, la función de evaluación (`_utilidad`) y los tres órdenes de acciones (`estandar`, `inverso`, `heuristico`), además de la integración en la interfaz (formulario adversarial, botones «Evaluar estado»/«Evaluar y jugar» y comparación `MINIMAX vs ALFA_BETA`).
- **Validación humana:** El equipo diseñó los escenarios 7–10, añadió `tests/test_minimax.py` (5 pruebas) y contrastó los resultados con la teoría: equivalencia de valor/acción entre alfa-beta y Minimax (Hipótesis 6) e independencia del orden de acciones (Hipótesis 7). También verificó que las utilidades terminales (± 100000) dominan la heurística y que los estados terminales rechazan cualquier evaluación.

J. Extensión de los reportes experimentales

- **Intervención de IA:** Se extendió `experimentos/ejecutar_experimentos_busqueda.py` para cubrir además los casos robustos `R01–R08` y los adversariales 7–10, con promedio de 5 corridas y secciones independientes en el reporte.
- **Validación humana:** El equipo re-ejecutó el script completo, revisó las tres secciones generadas y corrigió en el manual las cifras desactualizadas (tiempos de los casos 1–5 y criterio de éxito de Beam con `k=2`).

16.4 Metodología de validación humana del código generado por IA

Para cada contribución de IA se siguió el siguiente protocolo de verificación:

```
1. LECTURA: El estudiante leyó el código generado línea por línea antes de
   incorporarlo.
2. RAZONAMIENTO: Se razonó en voz alta (revisión en pareja) la lógica del
   código, identificando precondiciones, postcondiciones y casos borde.
3. PRUEBA UNITARIA: Se ejecutó la suite de pruebas existente. Si el
   nuevo código introducía regresiones, se depuró hasta restaurar los
   tests en verde (17 al inicio del proyecto; 25 en la versión actual).
4. PRUEBA MANUAL: Para componentes visuales o escenarios, se ejecutó
   la interfaz y se verificó el comportamiento esperado en al menos 3
   escenarios distintos.
5. DOCUMENTACIÓN: Se añadió o actualizó el docstring del componente
   marcando "AI usage: Yes" con descripción concreta de la intervención
   y "Student validation:" con la descripción de la prueba realizada.
```

16.5 Balance de contribuciones: Humano vs. IA

Componente	Contribución Principal	Nivel de Supervisión Humana
Diseño de la arquitectura en capas	Humano (equipo)	—
Reglas del juego (R1–R7) y lógica de estado	Humano (equipo) + IA (estructura)	Alta: pruebas unitarias exhaustivas
Algoritmos BFS y DFS	IA (esqueleto) + Humano (ajuste)	Alta: comparación manual de resultados
Algoritmos UCS y A*	IA (esqueleto) + Humano (ajuste)	Alta: verificación formal de optimalidad
Diseño de heurísticas (`h_manhattan` , `h_propia`)	IA (propuesta) + Humano (demostración matemática)	Muy alta: demostración formal + pruebas
Beam Search	IA (esqueleto) + Humano (parámetro `k`)	Alta: pruebas con `k` variable
Minimax / Poda alfa-beta (`algoritmos/minimax.py`)	IA (esqueleto y función de evaluación) + Humano (escenarios 7–10)	Alta: 5 pruebas de regresión + contraste con la teoría
Integración adversarial en la interfaz	IA (mayoritario) + Humano (prueba manual)	Media: validación de rechazos de estados terminales y del panel de comparación
Interfaz gráfica (Tkinter)	IA (mayoritario)	Media: validación visual interactiva
Escenarios de prueba JSON	IA (generación) + Humano (diseño pedagógico)	Alta: ejecución de todos los escenarios
Suite de pruebas unitarias	Humano (equipo)	—
Documentación técnica	IA (redacción, fórmulas) + Humano (revisión, aprobación)	Alta: revisión párrafo a párrafo

16.6 Limitaciones identificadas del código generado por IA

Durante el proceso de validación el equipo detectó y corrigió las siguientes situaciones que la IA no manejó correctamente en primera instancia:

1. **Heurística Euclidiana:** La IA implementó `h_euclidiana` en `a_estrella.py` como tercera heurística. Revisión posterior: no se expone en el desplegable de la interfaz (`HEURISTICAS = ("manhattan", "propia")`), porque en cuadrículas ortogonales euclidiana escala más bajo que Manhattan y no aporta información adicional. Es accesible por código para comparaciones.
2. **Ruta de escritura del generador de casos:** El script `generar_casos_minimos.py` escribía en la carpeta `casos_minimos` (guion bajo) mientras la carpeta versionada usa `casos minimos` (espacio). Fue detectado en la revisión del código y sigue sin corregirse (véase §15.2).
3. **Completitud de Beam Search con `k` pequeño:** Los primeros borradores del código de Beam Search no retornaban correctamente un `ResultadoBusqueda` con `exito = False` cuando el haz quedaba vacío; el equipo lo corrigió añadiendo el retorno apropiado al finalizar el bucle.
4. **Criterio de la prueba de Beam con `k=2` :** la prueba asumía que `k=2` sí hallaba ruta en `caso05_beam_search.json` . La re-ejecución del 2026-10-20 demostró que el haz estrecho queda atrapado en la copa; el equipo corrigió **la prueba y este manual** para alinearlos con el comportamiento medido, sin tocar el algoritmo.
5. **`experimentos/ejecutar_casos.py` :** el script de «esperado vs. obtenido» fue generado importando módulos que nunca existieron en el repositorio (`algoritmos.adversarial` , `verificacion.verificador_adversarial`), por lo que no se puede ejecutar. El equipo lo documentó como deuda técnica en lugar de darlo por válido.

16.7 Declaración de responsabilidad y autoría

Los integrantes del equipo declaramos que:

1. Comprendemos el funcionamiento de cada componente del sistema, incluyendo los generados con apoyo de IA.
2. Somos capaces de explicar, depurar y extender cualquier fragmento de código del repositorio.
3. El uso de IA se realizó de manera transparente, documentada y dentro de los términos del curso de Sistemas Inteligentes I.
4. La responsabilidad académica e intelectual del proyecto recae íntegra y exclusivamente en el equipo estudiantil.

Integrante	Firma de responsabilidad
Sophia Cardona	✓ Revisado y aprobado
Danna Alexandra Madrid Roa	✓ Revisado y aprobado
Michel Dahiana Arenas López	✓ Revisado y aprobado

17. Anexos

17.1 Anexo A — Diagramas solicitados

A.1 Diagrama de clases

```
[PENDIENTE DE IMPLEMENTACIÓN: insertar diagrama de clases]
Elementos a representar:
- Mapa (frozen) — filas, columnas, terreno, tipos
- Escenario (frozen) — mapa, bases, recurso, unidades, turno, portador,
  profundidad_minimax, prueba
- Unidad (frozen), Estado (frozen), Accion (frozen)
- Reglas — acciones_legales, costo, aplicar, ganador, es_terminal
- ProblemaNavegacion — estado_inicial, es_objetivo, sucesores
- ResultadoBusqueda (mutable)
- App(tk.Tk), Tablero(tk.Canvas)
```

A.2 Diagrama de dependencias

```
[PENDIENTE DE IMPLEMENTACIÓN: insertar diagrama de dependencias]
Fuente verificable: tabla de la sección 6.2.
```

A.3 Diagrama de secuencia de una búsqueda

```
[PENDIENTE DE IMPLEMENTACIÓN: insertar diagrama de secuencia]
Secuencia verificable:
App._ejecutar_busqueda
  -> App._problema
  -> App._correr
     -> ejecutar_algoritmo
        -> algoritmo registrado (BFS/DFS/UCS/A*)
           -> ProblemaNavegacion.sucesores
              -> Mapa.vecinos / Mapa.costo
     -> verificar_camino
     -> historial.append
  -> App._mostrar_resultado
  -> (si A*) App._comparar_heuristicas
  -> App._refrescar_estadisticas
```

A.4 Diagrama de estados del juego

```
[PENDIENTE DE IMPLEMENTACIÓN: insertar diagrama de estados]
Transiciones verificables: Reglas.aplicar sobre Estado
Acciones: MOVER (4 direcciones ortogonales) y ESPERAR
Condiciones terminales: Reglas.ganador
```

17.2 Anexo B — Ejemplo real de escenario JSON

Tomado de `escenarios/ejemplo.json` (contenido íntegro):

```
{
  "version": "1.0",
  "mapa": {"filas": 4, "columnas": 4},
  "tipos_terreno": {
    "camino":  {"costo": 1, "transitable": true},
    "pasto":   {"costo": 2, "transitable": true},
    "bosque":  {"costo": 4, "transitable": true},
    "pantano": {"costo": 7, "transitable": true},
    "muro":    {"transitable": false}
  },
  "terreno": [
    ["camino", "camino", "pasto", "pasto"],
    ["camino", "muro", "bosque", "pasto"],
    ["camino", "camino", "pantano", "pasto"],
    ["pasto", "pasto", "camino", "camino"]
  ],
  "bases": {"A": {"fila": 0, "columna": 0}, "B": {"fila": 3, "columna": 3}},
  "recurso": {"fila": 2, "columna": 2},
  "unidades": [
    {"id": "A1", "bando": "A", "tipo": "estandar", "fila": 0, "columna": 1},
    {"id": "A2", "bando": "A", "tipo": "estandar", "fila": 1, "columna": 0},
    {"id": "B1", "bando": "B", "tipo": "estandar", "fila": 3, "columna": 2},
    {"id": "B2", "bando": "B", "tipo": "estandar", "fila": 2, "columna": 3}
  ],
  "turno": "A",
  "juego": {"portador_recurso": null, "profundidad_maxima_minimax": 4},
  "prueba": {"modo": "busqueda", "unidad_inicio": "A1", "objetivo": {"fila": 3, "columna": 2}}
}
```

17.3 Anexo C — Ejemplo real de resultado serializado

Esquema producido por `ResultadoBusqueda.a_dict` (tipos, no valores concretos):

```
{
  "algoritmo": str,
  "exito": bool,
  "camino": list[list[int]],
  "costo": número | null,
  "estados_generados": int,
  "estados_expandidos": int,
  "maximo_frontera": int,
  "tiempo_ms": float
}
```

> `a_dict` redondea `tiempo_ms` a 4 decimales y convierte cada celda del camino a lista. Los valores concretos dependen del escenario y del algoritmo; no se transcriben resultados no reproducidos en esta elaboración.

17.4 Anexo D — Ejemplo real de código

Función de sucesores (`juego/problema_navegacion.py` , líneas 17–24):

```python
def sucesores(self, s):
    """Purpose: generar (nombre_accion, estado_siguiente, costo) en orden determinista.
    Preconditions: s es una celda transitable del mapa.
    Postconditions: solo celdas dentro del mapa y transitables; costo > 0.
    Complexity: O(1).
    AI usage: Yes. AI intervention: estructura del método.
    Student validation: (completar)."""
    return [(n, (f, c), self.mapa.costo(f, c)) for n, f, c in self.mapa.vecinos(*s)]
```

17.5 Anexo E — Evidencias de ejecución

E.1 Salida de pruebas automatizadas (Suite Unittest)

Ejecución verificada en consola con Python 3.14.0 (re-ejecución del 2026-10-20):

```
$ python -m unittest discover -s tests -v
test_k_1_2_4_8_devuelven_camino_valido_en_caso5 (test_beam_search.TestBeamSearch.test_k_1_2_4_8_devuelven_camino_valido_en_caso5) ... ok
test_sin_solucion_termina_sin_camino (test_beam_search.TestBeamSearch.test_sin_solucion_termina_sin_camino) ... ok
test_caso10_profundidades_1_a_4_limitan_busqueda (test_minimax.TestBusquedaAdversarial.test_caso10_profundidades_1_a_4_limitan_busqueda) ... ok
test_caso7_intercepta_portador_adyacente (test_minimax.TestBusquedaAdversarial.test_caso7_intercepta_portador_adyacente) ... ok
test_caso8_minimax_y_alfa_beta_equivalentes (test_minimax.TestBusquedaAdversarial.test_caso8_minimax_y_alfa_beta_equivalentes) ... ok
test_caso9_orden_cambia_trabajo_no_valor (test_minimax.TestBusquedaAdversarial.test_caso9_orden_cambia_trabajo_no_valor) ... ok
test_rechaza_profundidad_invalida (test_minimax.TestBusquedaAdversarial.test_rechaza_profundidad_invalida) ... ok
test_costo_no_positivo (test_paso1_2.TestCarga.test_costo_no_positivo) ... ok
test_dimensiones_inconsistentes (test_paso1_2.TestCarga.test_dimensiones_inconsistentes) ... ok
test_errores_multiples (test_paso1_2.TestCarga.test_errores_multiples) ... ok
test_unidad_sobre_muro (test_paso1_2.TestCarga.test_unidad_sobre_muro) ... ok
test_valido (test_paso1_2.TestCarga.test_valido) ... ok
test_acciones_y_sucesor (test_paso1_2.TestJuego.test_acciones_y_sucesor) ... ok
test_estado_compuesto (test_paso1_2.TestJuego.test_estado_compuesto) ... ok
test_hashable_e_inmutable (test_paso1_2.TestJuego.test_hashable_e_inmutable) ... ok
test_interceptar (test_paso1_2.TestJuego.test_interceptar) ... ok
test_no_atraviesa_muro_ni_ocupadas (test_paso1_2.TestJuego.test_no_atraviesa_muro_ni_ocupadas) ... ok
test_recoger_recurso (test_paso1_2.TestJuego.test_recoger_recurso) ... ok
test_unidad_que_empieza_sobre_el_recurso_lo_recoge (test_paso1_2.TestJuego.test_unidad_que_empieza_sobre_el_recurso_lo_recoge) ... ok
test_victoria (test_paso1_2.TestJuego.test_victoria) ... ok
test_caso2_bfs_vs_ucs_diferencia (test_paso3_ucs.TestUCS.test_caso2_bfs_vs_ucs_diferencia)
Caso 2: Ruta corta vs. ruta económica. ... ok
test_ucs_ejecucion_y_verificacion (test_paso3_ucs.TestUCS.test_ucs_ejecucion_y_verificacion) ... ok
test_admisibilidad_heuristicas (test_paso4_a_estrella.TestAEstrella.test_admisibilidad_heuristicas) ... ok
test_caso3_obstaculo_y_heuristica (test_paso4_a_estrella.TestAEstrella.test_caso3_obstaculo_y_heuristica)
Caso 3: La heurística intenta avanzar en línea recta, pero un muro ... ok
test_caso4_comparacion_heuristicas (test_paso4_a_estrella.TestAEstrella.test_caso4_comparacion_heuristicas)
Caso 4: Comparar A* con Manhattan vs Propia en el mismo escenario. ... ok

----------------------------------------------------------------------
Ran 25 tests in 0.153s

OK
```

(Versión inicial de este anexo: `Ran 17 tests in 0.017s ... OK` con Python 3.12.0; los 8 tests adicionales corresponden a `test_beam_search.py` y `test_minimax.py`, más la prueba `test_unidad_que_empieza_sobre_el_recurso_lo_recoge` que faltaba listar.)

E.2 Ejecución del generador de resultados cuantitativos

Ejecución automatizada mediante `python experimentos/ejecutar_experimentos_busqueda.py`:
Los resultados completos y tabulados (casos mínimos 1–5, robustos `R01-R08` y adversariales 7–10) se documentan en la **Sección 13** de este manual técnico y quedan guardados en el reporte `documentacion/resultados_experimentos_busqueda.md`. Todos los caminos obtenidos fueron comprobados por las 6 reglas del módulo `verificacion/verificador.py` (los casos sin ruta se reportan con `–` y los no transitables como `Rechazado`). La salida de consola termina con:

```
[OK] Resultados guardados exitosamente en:
  C:\...\TacticalGrid\documentacion\resultados_experimentos_busqueda.md
```

Complementariamente, `python experimentos/ejecutar_casos_adversariales.py` imprime por consola los mismos casos 7–10 en formato legible.

17.6 Anexo F — Inventario de escenarios

Archivo	Contenido verificable	`prueba.modo`
`escenarios/ejemplo.json`	Mapa 4×4; 4 unidades; recurso en `(2,2)`	`busqueda`

Archivo	Contenido verificable	`prueba.modo`
`escenarios/prueba_busqueda_dfs.json`	Mapa 4×4; algoritmo DFS; objetivo `(3,0)`	`busqueda`
`escenarios/prueba_partida.json`	Mapa 4×4; sin portador inicial	`partida`
`escenarios/prueba_adversarial.json`	Mapa 4×4; portador `B1`	`adversarial`
`escenarios/invalido_a_proposito.json`	Costo `0` en `pasto` y tipo `lava` inexistente	No aplica (inválido)
`escenarios/casos minimos/caso01_bfs_vs_dfs.json`	Mapa 20×20; BFS vs DFS	`busqueda`
`escenarios/casos minimos/caso2_ruta_corta_vs_economica.json`	Mapa para BFS vs UCS	`busqueda`
`escenarios/casos minimos/caso3_obstaculo_y_heuristica.json`	Mapa con obstáculo	`busqueda`
`escenarios/casos minimos/caso4_comparacion_heuristicas.json`	Mapa para comparar heurísticas	`busqueda`
`escenarios/casos minimos/caso05_beam_search.json`	Referencia `BEAM_SEARCH`, `k=1`	`busqueda`
`escenarios/casos minimos/caso06a_sin_recurso.json`	Estado compuesto sin portador	`adversarial`
`escenarios/casos minimos/caso06b_con_recurso.json`	Estado compuesto con portador `A1`	`adversarial`
`escenarios/casos minimos/caso07_decision_adversarial.json`	Portador rival `B1` adyacente; profundidad 3	`adversarial`
`escenarios/casos minimos/caso08_minimax_vs_alfa_beta.json`	Par Minimax vs Alfa-Beta	`adversarial`
`escenarios/casos minimos/caso09_orden_acciones.json`	Orden de acciones `estandar` / `inverso` / `heuristico`	`adversarial`
`escenarios/casos minimos/caso10_profundidad.json`	Profundidad 1–4	`adversarial`
`escenarios/casos_invalidos/inv01_dimensiones_inconsistentes.json`	Dimensiones incoherentes	No aplica (inválido)
`escenarios/casos_invalidos/inv02_terreno_inexistente.json`	Terreno no declarado	No aplica (inválido)
`escenarios/casos_invalidos/inv03_costo_cero.json`	Costo no positivo	No aplica (inválido)
`escenarios/casos_invalidos/inv04_base_fuera_del_mapa.json`	Base fuera de límites	No aplica (inválido)
`escenarios/casos_invalidos/inv05_unidad_sobre_muro.json`	Unidad sobre obstáculo	No aplica (inválido)
`escenarios/casos_invalidos/inv06_id_repetido.json`	Identificador duplicado	No aplica (inválido)
`escenarios/casos_invalidos/inv07_bando_invalido.json`	Bando distinto de A/B	No aplica (inválido)
`escenarios/casos_invalidos/inv08_turno_invalido.json`	Turno distinto de A/B	No aplica (inválido)
`escenarios/casos_invalidos/inv09_portador_inexistente.json`	Portador inexistente	No aplica (inválido)
`escenarios/casos_invalidos/inv10_json_mal_formado.json`	JSON truncado	No aplica (inválido)
`escenarios/casos_invalidos/inv11_falta_campo_obligatorio.json`	Falta `bases`	No aplica (inválido)
`escenarios/casos_invalidos/inv12_recurso_fuera_del_mapa.json`	Recurso fuera de límites	No aplica (inválido)
`escenarios/casos_robustos/r01_meta_inalcanzable.json`	Meta encerrada por muros	`busqueda`
`escenarios/casos_robustos/r02_inicio_igual_meta.json`	Origen igual a meta	`busqueda`
`escenarios/casos_robustos/r03_corredor_1x20.json`	Mapa 1×20	`busqueda`
`escenarios/casos_robustos/r04_unidad_encerrada.json`	Unidad sin vecinos transitables	`busqueda`
`escenarios/casos_robustos/r05_objetivo_en_muro.json`	Objetivo sobre muro	`busqueda`
`escenarios/casos_robustos/r06_costos_fraccionarios.json`	Autopista de costo `0.5`	`busqueda`
`escenarios/casos_robustos/r07_dimensiones_y_nombres_nuevos.json`	Mapa 8×12 y terrenos nuevos	`busqueda`
`escenarios/casos_robustos/r08_mapa_grande_40x40.json`	Mapa 40×40	`busqueda`
`escenarios/casos_robustos/r09_portador_en_su_base.json`	Victoria en estado inicial	`partida`
`escenarios/casos_robustos/r10_unidad_sobre_el_recurso.json`	Unidad sobre el recurso	`partida`

17.7 Anexo G — Glosario

Término	Definición en el contexto del proyecto
Estado	Tupla inmutable `(unidades, recurso_pos, portador, turno)`
Acción	`MOVER` a una celda ortogonal o `ESPERAR`
Sucesor	Celda adyacente transitable con su acción y costo
Frontera	Colección de estados pendientes de expandir
Expandido	Estado cuyos sucesores fueron analizados
Generado	Estado insertado en la frontera
c_min	Menor costo entre terrenos transitables (`Mapa.costo_minimo`)
Portador	Unidad que lleva el recurso
Interceptación	Entrada a la celda del portador enemigo, que lo elimina
Escenario válido	JSON que supera `validar_datos` sin errores

Nota final de alcance documental

Este manual describe el estado del repositorio en el commit `ae0664c` de la rama `main` , más los cambios sin commitear señalados en la portada y en la sección 15.2. Cualquier cambio posterior en el código puede invalidar total o parcialmente las referencias de línea y los comportamientos aquí descritos. Las secciones marcadas con `[PENDIENTE DE IMPLEMENTACIÓN]` , `[COMPLETAR CUANDO SE IMPLEMENTE]` o `[AGREGAR EVIDENCIA]` requieren completarse con información o artefactos que no existen en el repositorio al momento de la elaboración.